SE(3)-WAM: Role-Aware 4D World–Action Modeling for Dynamic Manipulation

Anonymous CVPR submission

Figure placeholder: Fig. 1 teaser — a role-aware 4D view of a moving scene (real robot)

Layout: full width, two parts. **(a) Left, ∼62%:** one real-robot frame of the *linear-motion grasp* at the instant a chunk is issued. Overlays in the paper's semantic colors: blue = the target's past SE(3) poses from the causal tracker (small triads at $-350 \ldots 0$ ms, fading with age); orange = the target's SE(3) forecast at the execution times of the chunk, taken from the training-time heads evaluated offline; light orange arrows = sparse 3D flow on the 14×14 grid over the whole scene; purple = the commanded gripper path meeting the target at τ^{a}. A gray ghost marks the pose observed at capture, with a dashed red arrow showing where a reactive policy would close. A thin timeline strip under the image: capture $\tau_t \rightarrow$ latency $\ell_t \rightarrow$ execution τ^{a}. **(b) Right, ∼38%:** top: "Image-space WAM: future frames or latents at uniform fidelity" (filmstrip icon); middle: "SE(3)-WAM: SE(3) for the target, sparse 3D flow for the whole scene, training only" (triads + sparse arrows); bottom **(required)**: SR vs. added latency (0–200 ms) for SE(3)-WAM and the strongest baseline (data: the latency sweep of Appendix C.2). **Rules:** real photo kept unretouched; overlays drawn from logged tracker/forecast/command data of that trial; caption states the trial's outcome.

Figure 1. **A role-aware 4D view of a moving scene.** Between image capture and the first command, a part on the conveyor moves farther than the grasp tolerates. SE(3)-WAM encodes the target's metric motion (blue) and is trained to forecast the target's SE(3) pose precisely and the whole scene coarsely as a sparse 3D flow (orange), so that its commands meet the target when they execute (purple). Image-space world–action models forecast appearance at uniform fidelity instead. Right: success against added latency on Dynamic-14.

Abstract

Manipulating moving objects requires actions that fit the scene when they execute, not when the image was captured, yet every millisecond spent inferring the scene's motion lets the target move further. World–action models forecast future images or latents at uniform fidelity, spending time on appearance while leaving the target's metric pose implicit; target-centric predictors are fast but ignore the surroundings that constrain or drive the motion. We observe that the parts of a dynamic scene need different precision: the target's pose decides contact to within millimeters, the bodies around it matter only through the clearance they leave and the motion they impart to the target, and the remaining background does not matter. We present SE(3)-WAM, a world–action model built on this role-aware 4D structure. It encodes the target's motion as a metric SE(3) pose history and is trained to forecast the target's future SE(3) trajectory precisely and the whole scene coarsely as a sparse 3D flow. The forecasts serve only as training supervision and are skipped at deployment, so inference needs neither past frames nor a generated future and stays fast enough for real-time control. On Dynamic-14, a simulated benchmark in which the scene keeps moving during inference, SE(3)-WAM achieves [XX.X]% success, [XX.X] points above the strongest baseline, and succeeds in [XX]% of trials on six real-robot tasks with diverse target motion.

1. Introduction

Learning-based manipulation has advanced rapidly, from diffusion policies to vision–language–action (VLA) and world–action models (WAMs) [6, 7, 12, 21, 43, 49, 63], yet most of this progress assumes that objects stay still unless the robot moves them. In factories, kitchens, and human–robot handovers, objects keep moving while the robot perceives, computes, and acts. This poses two coupled difficulties. A single image does not reveal how the target is moving, and by the time an action executes, the target is no longer where it was seen: a part on a conveyor at 0.1 m/s travels 15 mm during a 150 ms delay, more than a parallel-jaw grasp typically tolerates (Fig. 1).

Recent work equips policies with a memory of past motion and a forecast of the future [14, 28, 39, 56]. Both cost the resource that dynamic manipulation lacks most, time: richer histories and denser forecasts add computation, and the added latency enlarges the very error the forecast is meant to remove. Existing designs therefore trade information against speed. Video-based WAMs forecast whole future frames or latents at uniform fidelity [43, 49], also when conditioned on motion cues, which remain in image space and do not separate the target from other moving regions [28], and methods that restrict prediction to dynamic regions or salient tokens still predict in image space [39, 56]. In both, most capacity goes to appearance, and the quantity that decides contact, the target's metric pose when the ac-

tion executes, stays implicit. Target-centric predictors and dynamic VLAs with object-level forecasts sit at the other end [14, 18, 29]: they are fast but blind to surroundings that can block or redirect the motion. None of them provides knowledge of the moving scene that is precise where contact depends on it, coarse elsewhere, and cheap to use at execution time.

Our key observation is that a dynamic policy needs to know very little precisely. Contact depends on the target's pose at the moment the action executes, to within millimeters. The bodies around it matter only through the clearance they leave and the motion they impart to the target, which a coarse estimate captures, and the rest of the background does not matter. Nor does this knowledge have to be computed at test time: a forecast that supervises training shapes what the policy represents and can be dropped at deployment [53]. SE(3)-WAM builds on both ideas (Fig. 2). It sees the current image together with short metric SE(3) histories of the target and the robot, which convey motion directly instead of a stack of past frames. A world stream is trained to forecast, in one metric 4D space, the target's future SE(3) trajectory precisely and the whole scene coarsely as a sparse 3D flow, and an action stream reads the world stream's summary and replans as often as its latency permits. SE(3)-WAM thus gains the precision of a metric forecast at a latency close to that of a reactive policy.

We evaluate SE(3)-WAM on Dynamic-14, a simulated benchmark of fourteen tasks in which observation, scene motion, and action execution share one physical clock, so the scene keeps moving while a policy computes. SE(3)-WAM achieves state-of-the-art success of [XX.X]%, [XX.X] points above the strongest baseline, at a p95 latency of [XX] ms, and succeeds in [XX]% of trials on six real-robot tasks, against [XX]% for the strongest baseline; it also leads on the 24 tasks of DOMINO [14]. Five final-scale controls isolate the contributions of target-motion history, future supervision, and its two geometric components, and compare role-aware with uniform-fidelity forecasts.

Our contributions are threefold:

- We propose role-aware 4D modeling for dynamic manipulation. Tracing how latency and estimation errors reach the contact shows that only the target needs millimeter precision, so we represent the target precisely by its metric SE(3) pose history and future trajectory, and the whole scene coarsely by a sparse 3D flow, in metric 3D rather than in image space.
- We present SE(3)-WAM, a world–action model trained with these role-aware 4D forecasts as supervision, at a latency close to that of a reactive policy.
- We build Dynamic-14, in which the scene keeps moving during inference, and show state-of-the-art results on it, on DOMINO, and on six real-robot tasks.

2. Related Work

Dynamic manipulation. Compensating for the time between sensing and acting is a classical problem, from control with dead time [36] to catching systems that track the target with Kalman-type filters and plan an interception [2–4, 31]; such systems are precise but engineered for each object and task. Learned policies have recently been brought to moving scenes [1]. PUMA reads historical optical flow and is supervised with future object features [14], DynamicWAM conditions a WAM on flow and kinematic descriptors and executes it with real-time chunking [28], and other methods forecast salient tokens, future images, or compact future codes and feed them to the policy at test time [29, 39, 54, 57]; DynaWM refines the chunks of a base VLA for moving objects [11], and DynamicManip synthesizes dynamic demonstrations from a static one [23]. Execution schemes instead reduce or hide latency, by overlapping inference with execution [34, 47], keeping consecutive chunks consistent [8, 26], or conditioning on the execution time [40]. They decide when actions are computed and applied, whereas SE(3)-WAM changes what the policy represents; we run every method under the same real-time executor and, as a control, with its original executor. DOMINO [14] and PhysMani-Bench [54] provide diverse dynamic-manipulation tasks, but their released default evaluation pipelines advance the simulation only after a policy has returned its prediction; Dynamic-14 advances the scene during inference and charges each method its measured latency.

Predictive supervision in world–action models. World–action models (WAMs) generate future video and actions in one network [22, 43, 49, 61], building on video prediction as a policy objective [41, 46]. Because video is costly and dominated by appearance, recent work changes what is forecast, such as dynamic regions, depth, and semantic features [56], flow and geometry alongside RGB [52], object masks or slots [27, 50], or compact latents [60], and asks whether the forecast is needed at test time: Fast-WAM retains most of the benefit of video modeling without test-time imagination [53], and PUMA and others drop their predictors after training [14, 38, 58]. SE(3)-WAM adopts this training-only use and changes the target of the supervision: instead of pixels, features, or latents at one resolution, a metric 4D state whose precision follows each entity's role in contact.

Geometric representations of motion. Metric 3D inputs make policies more precise and sample-efficient [33, 55]. Motion has been represented by 2D point tracks [5, 45, 48], 3D flow [16, 19, 51], and object keypoints or SE(3) trajectories [18, 30, 37], and rigid-body decompositions have long been used to learn scene dynamics [9, 10]. These works describe the scene with one kind of representation, dense

flow or tracks everywhere or a pose for each object; DOCIR separates the robot, the target, and obstacles, but as input encodings [13]. SE(3)-WAM assigns precision by role: a coarse sparse 3D flow describes the whole scene, and the target, on which contact depends to within millimeters, is additionally described by its SE(3) history and forecast.

3. SE(3)-WAM

SE(3)-WAM is a world–action model that represents a dynamic scene at role-dependent precision (Fig. 2). It encodes the current RGB-D frame, whose RGB a frozen MAE ViT-B/16 [17] turns into 196 patch tokens and whose depth gives each patch the metric 3D position of its center point in the robot base frame, together with short histories of the target's SE(3) pose and of the robot state. These 212 scene tokens and 16 learnable summary tokens that aggregate them form a world stream, which forecast heads supervise during training with the target's future SE(3) trajectory and a sparse 3D flow of the whole scene. An action stream shares attention with the world stream, reads its summary tokens, and generates action chunks by conditional flow matching [24].

3.1. Problem Formulation

SE(3)-WAM learns a policy from demonstrations. At query t, the policy receives an RGB image, a depth map, and the robot state captured at time τ_t, together with short histories of the robot state and of the pose $T(\tau) = (R(\tau), \mathbf{p}(\tau)) \in \mathrm{SE}(3)$ of a target object in the robot base frame. It outputs a chunk $(\mathbf{a}_{t,0}, \dots, \mathbf{a}_{t,H-1})$ of commands, one per control period Δ_a, indexed by time from capture: the j-th command is meant for

$$\tau^{\mathrm{a}}_{t,j} = \tau_t + j\Delta_a + d_{\mathrm{task}}. \tag{1}$$

Here d_{task} is a task's built-in command delay. The chunk becomes available only after a latency ℓ_t, so its commands with $j\Delta_a < \ell_t$ have expired, and the later ones execute at their times $\tau^{\mathrm{a}}_{t,j}$. A command must therefore fit the target's pose at $\tau^{\mathrm{a}}_{t,j}$ rather than at τ_t, a pose that the policy must estimate from observations at τ_t, and the lead $\Delta = \tau^{\mathrm{a}}_{t,j} - \tau_t$ of every executed command is at least ℓ_t, so it grows with every millisecond of inference. The bodies of a scene play different roles. The *target* is the body whose pose must be matched within a contact or containment tolerance ε, such as an object to grasp or a receptacle to place into. The *context* consists of the bodies that constrain or drive the target's motion: obstacles, such as an occluder or a wire, must be avoided with a clearance m, and drivers, such as a conveyor, a ramp, or a striker, move the target. The rest of the scene does not affect the task. Sec. 3.2 traces how the error of this estimate depends on the lead and on these roles, and derives the representation of SE(3)-WAM from it.

3.2. Role-Aware 4D Representation

What must be precise. Let a policy act, explicitly or implicitly, on an estimate of the target pose at τ^{a} with position error e and rotation error θ. A contact point at lever arm r is then off by at most $e + 2r\sin(\theta/2)$. Writing the target's motion over the lead as a constant velocity $\mathbf{v}$ plus a residual (acceleration, rolling, impacts), and the lead that the policy assumes as $\hat{\Delta}$, gives

$$e \le e_{\mathrm{loc}} + \hat{\Delta}\, e_v + \|\mathbf{v}\|\, e_\tau + e_{\mathrm{dyn}}, \tag{2}$$

with the errors e_{loc} of the current position, e_v of the velocity, $e_\tau = |\hat{\Delta} - \Delta|$ of the timing, and e_{dyn} of the residual motion (Appendix A.1). The tolerance ε is typically a few millimeters, and the rotation error is amplified by the lever arm, so the target's pose and motion must be known precisely. The context needs far less precision. An obstacle only has to be avoided with the clearance m, which is usually far larger than ε, and a driver matters through the motion it imparts to the target, which then appears in the target's own future; in both cases, a coarse estimate of the context suffices (Fig. 3c). We therefore represent the target by metric SE(3) states, both its observed history and its forecast trajectory, and the whole scene by a sparse 3D flow, all in the robot base frame and in physical time.

What latency allows. Latency enters Eq. (2) twice, through the lead that multiplies every velocity error and through the motion that the residual accumulates. By Eq. (2), information that costs inference time pays off only if it removes more error than the added latency creates. SE(3)-WAM keeps its inputs cheap, with one current image and low-dimensional state histories instead of past frames, and uses its forecasts only to supervise training, so that they add no inference time (Sec. 3.4).

Target motion: history and forecast. A single pose does not reveal how the target moves (Fig. 3a). The target history covers the last 8 control steps (350 ms at 20 Hz). It consists of the current target pose in the base frame and the seven interval motions between consecutive samples, each with its translation, relative rotation, time interval, and finite-difference velocity and acceleration in physical units, and forms 8 target-state tokens (Appendix A.2). These tokens state directly the velocity and acceleration on which e_v and part of e_{dyn} depend, at the cost of a few low-dimensional tokens. The robot history adds 8 end-effector state tokens. In simulation, the target pose comes from the simulator; on the real robot, a causal tracker estimates it from the RGB-D stream (Appendix D).

The forecast describes where the target will be when the commands execute (Fig. 3b). It covers fixed physical times $\tau_t + \delta_h$, with $\delta_h = h{\cdot}50\,\mathrm{ms}$ and $h = 1, \dots, 8$, from 50 ms to

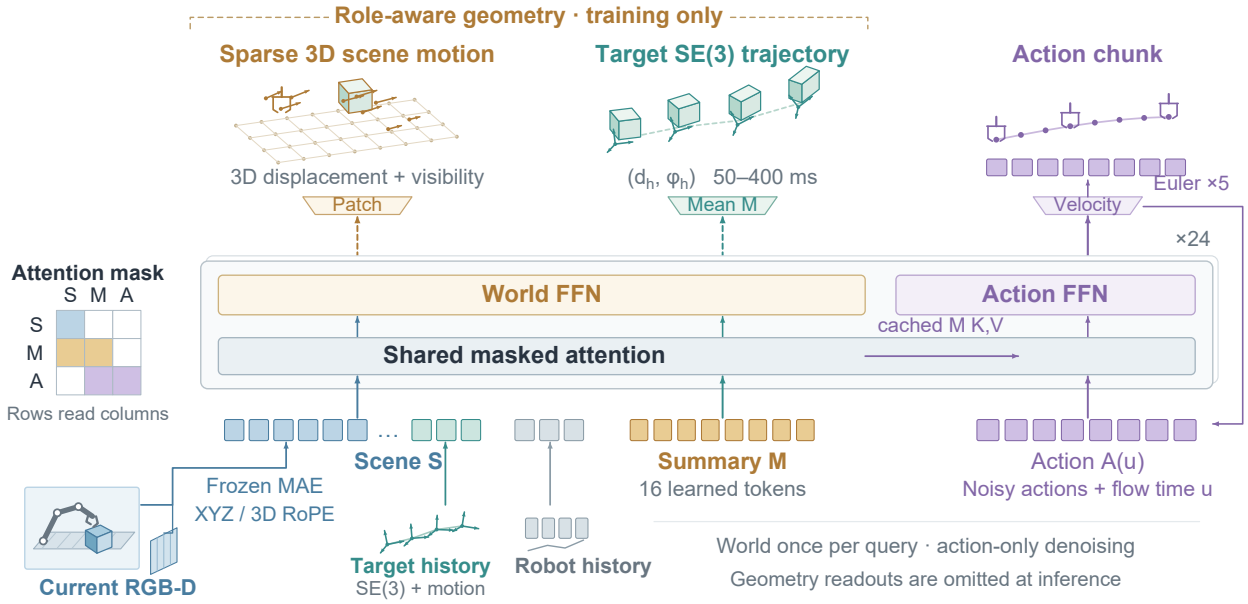


Figure 2. **SE(3)-WAM architecture.** Read from bottom to top. Current RGB-D and target/robot histories feed the world stream. World and action streams share attention and use separate FFNs. Training-only heads (dashed) predict sparse whole-scene 3D displacement and visibility from patch states, and target SE(3) trajectories from the mean summary. Actions read summary tokens through the structured mask. At inference, world keys and values are computed once per query and reused across 5 Euler steps; forecast heads are skipped. Geometry drawings are schematic.

400 ms after capture, so that every command time $\tau^{\mathrm{a}}_{t,j}$ with $j \geq 1$ is a forecast horizon. Like all our motion labels, it is cumulative from the current state, and every loss averages over valid labels only. For each horizon, the target's motion is

$$\mathbf{d}_h = \mathbf{p}(\tau_t{+}\delta_h) - \mathbf{p}(\tau_t), \quad \boldsymbol{\phi}_h = \mathrm{Log}\big(R(\tau_t{+}\delta_h)R(\tau_t)^\top\big)^\vee. \tag{3}$$

A linear pose head on the mean of the final summary tokens predicts $(\hat{\mathbf{d}}_h, \hat{\boldsymbol{\phi}}_h)$ with the loss

$$\mathcal{L}_P = \frac{1}{|\mathcal{V}|} \sum_{h \in \mathcal{V}} \Big[\|\hat{\mathbf{d}}_h - \mathbf{d}_h\|^2 + \lambda_R \, d_{\mathrm{SO}(3)}\big(\hat{R}_h, R_h\big)^2 \Big], \tag{4}$$

where $\hat{R}_h = \mathrm{Exp}(\hat{\boldsymbol{\phi}}_h^{\wedge}) R(\tau_t)$ and $\mathcal{V}$ is the set of valid horizons. Because the pose head reads the summary tokens, the only world tokens that the actions attend to (Sec. 3.3), the precise target supervision shapes exactly the representation that the actions use. The past and the future of the target thus share one metric frame and one time axis.

Scene motion: sparse 3D flow. The center points of the 196 patches on the 14×14 grid cover the target, the robot, and the surroundings. For each point i and horizon h, a linear head on its final patch token predicts the base-frame displacement $\hat{F}_{ih} \in \mathbb{R}^3$ and a visibility logit $\hat{o}_{ih}$, and

$$\mathcal{L}_F = \frac{1}{|\mathcal{V}_F|} \sum_{(i,h) \in \mathcal{V}_F} \Big[\tfrac{1}{3} \big\| (\hat{F}_{ih} - F_{ih})/s \big\|^2 + \mathrm{BCE}(\hat{o}_{ih}, o_{ih}) \Big], \tag{5}$$

with a fixed scale s fitted on the training labels, the visibility o_{ih}, and the set $\mathcal{V}_F$ of valid labels. The grid samples the workspace sparsely, about [X] mm between points. The flow thus gives the context and the robot a motion estimate at this spacing and stays near zero on the static background, while the target additionally receives the dedicated SE(3) supervision of Eq. (4). The flow supervises the patch tokens from which the summaries draw, one level below the precise target supervision; this is how SE(3)-WAM allocates precision by role.

Auxiliary supervision. Two further targets regularize the representation; we selected them on development scenes (Appendix E). Future appearance heads on the patch tokens predict, at every horizon, the metric depth, the features of a frozen DINOv3 teacher [35], and the RGB of moving surfaces. Separately, a small head on the summary tokens reconstructs the RGB of the current target region, which ties the summaries to the target without a future label. Their losses form $\mathcal{L}_{\mathrm{aux}}$ (Appendix A.2).

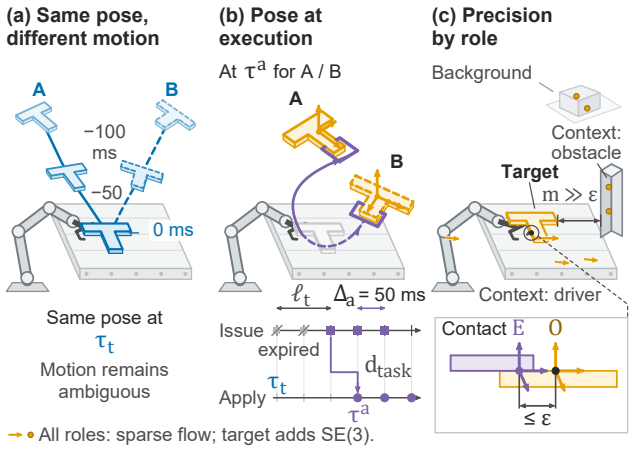


Figure 3. **Acting on a moving target** (schematic). (a) Different histories reach the same captured pose at τ_t. (b) At the same execution time τ^a, target translation and rotation differ, requiring different commands (purple). Issue slots are spaced by $\Delta_a = 50$ ms from capture. Gray slots expire during latency ℓ_t; retained commands apply after task delay d_{task}, first at $\tau^a = \tau_t + \ell_t + d_{\text{task}}$. (c) Contact aligns E and O within millimeter-scale ε; an obstacle leaves clearance $m \gg \varepsilon$, a conveyor drives the target, and distant background does not affect the task. Sparse flow covers all roles (arrows: displacement; dots: zero); the target additionally receives precise SE(3) supervision. Poses and delays are illustrative ($\ell_t = 2\Delta_a$, $d_{\text{task}} = \Delta_a$); contact is enlarged.

3.3. Action Generation and Joint Learning

Action stream. The action stream holds $H = 8$ tokens, one per command of the chunk; each embeds a noisy command, an end-effector translation and rotation increment plus a gripper command (Appendix A.2). The attention mask $\mathcal{G}$ lets scene tokens attend to the scene, summary tokens to the scene and the summaries, and action tokens to the summaries and the actions, so the actions read the world through the summary tokens and the world stream does not depend on the actions. In every layer, both streams share the attention projections, through which they exchange information, and keep separate feed-forward networks, which give each its own capacity; we selected this backbone over fully separate parameters on development scenes (Appendix E.4). World tokens use layer normalization, and action tokens use adaptive layer normalization conditioned on the flow time.

Joint learning. For a chunk $\mathbf{a}_1$ normalized with training statistics (Appendix A.2), noise $\mathbf{a}_0 \sim \mathcal{N}(0, I)$, and flow time $u \sim \mathcal{U}[0, 1]$, the action stream v_ψ, with parameters ψ, regresses the flow-matching velocity

$$\mathcal{L}_A = \mathbb{E} \left\| v_\psi(\mathbf{a}_u, u \mid \mathcal{S}_t, \mathcal{G}) - (\mathbf{a}_1 - \mathbf{a}_0) \right\|^2, \qquad (6)$$

with $\mathbf{a}_u = (1-u)\mathbf{a}_0 + u\mathbf{a}_1$, the scene tokens $\mathcal{S}_t$ of query t, and the error averaged over valid commands. All parts are trained jointly with

$$\mathcal{L} = \mathcal{L}_A + \lambda_P \mathcal{L}_P + \lambda_F \mathcal{L}_F + \lambda_{\text{aux}} \mathcal{L}_{\text{aux}}, \qquad (7)$$

with the weights given in Sec. 4.1.

3.4. Efficient Inference and Execution

SE(3)-WAM trains its forecast heads jointly and skips them at test time, as recent WAMs do [22, 53]. Because world tokens attend neither to the actions nor to the flow time, the world stream runs once per query, and the action stream integrates 5 Euler steps on its cached keys and values. The executor counts latency in whole control periods, as RTC does [8]. A query of total latency L_t hands its chunk over at the first control-period boundary at or after it ends, $\ell_t = k_t \Delta_a$ after capture with $k_t = \lceil L_t / \Delta_a \rceil$, and execution continues from command k_t, the one meant for that boundary. There the policy observes again and starts the next query; until then, the executor continues the previous chunk. The policy therefore replans as often as its latency allows, before every command when a query fits in one control period. Because each command is issued at the time its label refers to, the action stream needs no latency input, and the timing error e_τ of Eq. (2) is limited to actuation delays. Every executed command, at most $(H-1)\Delta_a + d_{\text{task}}$ after capture, lies within the horizons of the forecast supervision (Appendix B.3). On the robot, L_t also includes tracking the target in the newest frame.

4. Experiments

We evaluate how SE(3)-WAM performs when the scene keeps moving during inference (Sec. 4.2), what each of its components contributes (Sec. 4.3), and how it transfers to a real robot (Sec. 4.4).

4.1. Setup

Dynamic-14. Dynamic-14 is a simulated benchmark in which observation, scene motion, and action execution share one physical clock, so the scene keeps moving while a policy computes and every method pays for its own latency. It contains fourteen tasks in MuJoCo/robosuite [42, 62] in five families: grasping a moving target (harmonic, conveyor, translating, rotating, and occluded motion), placing into a moving receptacle, passing a wand along a rotating wire, capturing rolling or sliding objects, and grasping after an impact (Fig. 4; Appendix B). Experts with privileged simulator state provide 800 demonstrations per task. We select configurations on development scenes and report all Dynamic-14 results on 1,400 held-out scenes (100 per task, new seeds).

Evaluation. During each query, physics advances by the method's own inference latency, drawn from its latency distribution measured at batch size 1 on one RTX 5090, the

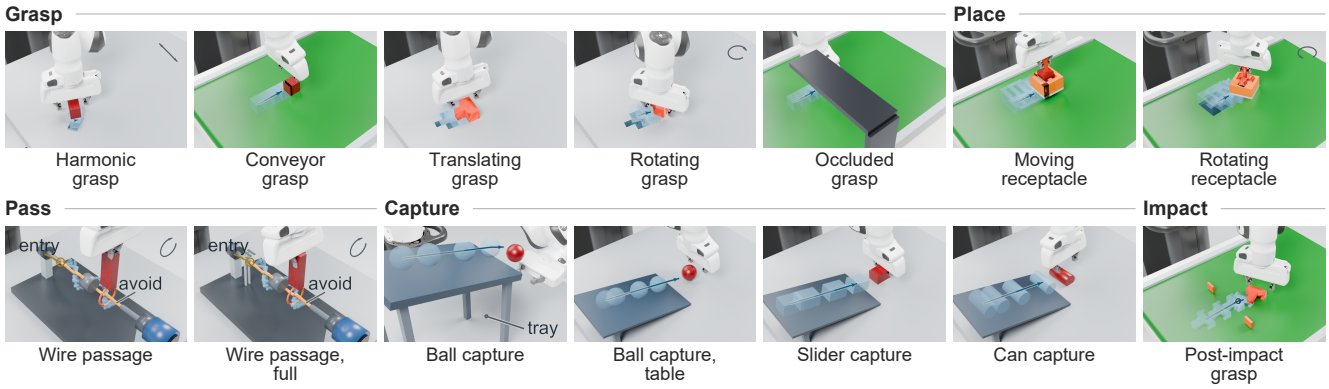


Figure 4. **Dynamic-14.** Fourteen tasks in five families, rendered from recorded simulator states with a common agent view. Four translucent blue poses fade from recent to earlier history; navy traces follow measured past motion. Gray corner arrows indicate rotation or oscillation qualitatively. A small ring marks the target center during recorded impact. Both Pass panels show passage: the wire is a target at entry and an obstacle along the path; Full additionally requires acquisition, exit and release. Occluded motion stays hidden. These native-expert examples illustrate task dynamics, not SE(3)-WAM outcomes; no future prediction is shown.

GPU of our robot, independently of the hardware that runs the rollouts (Appendix B.3). All methods share one executor that indexes each chunk by time from capture (Sec. 3.4): a query starts at a control-period boundary, and its chunk takes over at the first boundary at or after the query ends (Appendix B.3). Besides this native-latency setting, we add 150 ms of latency to every method and pause physics during inference; Appendix C.2 adds equal-latency and original-executor controls. We report the success rate (SR), averaged over the fourteen tasks with equal weight and over training seeds, and the p95 latency, the measured time from an observation to its first command (time-to-action). Paired 95% intervals come from a bootstrap over held-out scenes.

Baselines. We compare with reactive policies, dynamic VLAs, image- or latent-space WAMs, and a dynamic WAM (Table 1). Each baseline is trained on the same demonstrations with its official code, inputs, and recipe, including a rendered wrist camera and frame or flow histories where the recipe uses them (Appendix B.5). SE(3)-WAM uses one agent view with its depth and the target's SE(3) history, without past frames. To give baselines the same target-motion information, $\pi_{0.5}$ and DynamicWAM also run with our SE(3) history tokens (+SE(3)).

Implementation. The transformer has 24 layers of width 1024 with 16 heads and 523.8M trainable parameters; the MAE ViT-B/16 encoder and the DINOv3 teacher stay frozen. On Dynamic-14, it trains from random initialization for 20 epochs with AdamW, $\lambda_P = \lambda_F = 1$, and $\lambda_R = \lambda_{\text{aux}} = 0.1$ (Appendix A.3). On the real robot, it is fine-tuned per task from a 48-layer base pretrained on DROID [20] (Appendix A.4).

Table 1. **Main results** on 1,400 held-out Dynamic-14 scenes. SR (%) averages the fourteen tasks with equal weight and three training seeds for SE(3)-WAM (one for baselines), with each method's native latency, with 150 ms of added latency, and with physics paused during inference. p95 lat.: p95 latency, the time-to-action (ms) at batch size 1 on one RTX 5090; these latencies are replayed in simulation; rollout hardware is reported in Appendix C.2. All methods receive the current agent-view RGB and the robot state. Wrist: an additional wrist camera. History: past RGB frames (RGB), an optical-flow or flow-based motion history (Flow), or the target's SE(3) history. SE(3)-WAM additionally uses the depth of the current frame to give each patch its metric 3D position. DP: Diffusion Policy. Best in bold. Per-family and per-task results, contact errors, and intervals in Appendix C.1.

Method	History				SR (%)			
	Wrist	RGB	Flow	SE(3)	Native	+150 ms	Paused	p95 lat. (ms)
ACT [59]					42.0	24.1	40.9	10.7
DP [12]		✓			0.0	0.0	—	1701.9
$\pi_{0.5}$ [6]					—	—	—	—
$\pi_{0.5}$+SE(3)	—			✓	—	—	—	—
DynamicVLA [47]		✓			—	—	—	—
PUMA [14]			✓		—	—	—	—
DreamVLA [56]		✓			—	—	—	—
OpenWAM-α [43]					—	—	—	—
DynamicWAM [28]			✓		—	—	—	—
DynamicWAM+SE(3)	—		✓	✓	—	—	—	—
SE(3)-WAM				✓	—	—	—	—

4.2. Simulation Results

Comparison. With its native latency, SE(3)-WAM reaches [XX.X]% SR on held-out Dynamic-14 scenes (Table 1),

Table 2. **DOMINO results** on 24 single-arm clean-L1 tasks, each method trained on DOMINO's demonstrations (one training seed per method, 50 episodes per task and method). SR (%) is grouped by interaction type and averaged over tasks; MS is the manipulation score. Protocol: Appendix C.3.

Method	Intercept	Track	All	MS
$\pi_{0.5}$	—	—	—	—
DynamicWAM	—	—	—	—
PUMA	—	—	—	—
SE(3)-WAM	—	—	—	—

[X.X] points above the strongest baseline, [name], and [X.X] points above the strongest baseline that receives the same SE(3) history, [name]. Its gains are largest on [task types] (Appendix C.1). Its p95 latency of [XX] ms is [X]× lower than that of the image-space WAMs and close to that of [the reference reactive policy, to be fixed].

Latency and protocols. With 150 ms of added latency, SE(3)-WAM keeps [XX.X]% SR, whereas the reactive policies drop to [XX.X]% and the image-space WAMs to [XX.X]% on average, as Eq. (2) predicts: added latency lengthens the lead that multiplies every velocity error, so methods that do not anticipate the target's motion lose most. Fig. 1 (right) and Appendix C.2 give the full sweep from 0 to 200 ms. With physics paused during inference, which ignores the cost of computation, SE(3)-WAM still leads [name] by [X.X] points, against [X.X] points with native latency. When every method is charged the p95 latency of SE(3)-WAM, its margin is [X.X] points, and running [the baselines that rely on their own asynchronous execution] with their original executors changes their SR by at most [X.X] points (Appendix C.2).

DOMINO. We also train SE(3)-WAM, $\pi_{0.5}$, DynamicWAM, and PUMA on 24 single-arm Franka tasks of DOMINO [14] with its demonstrations and evaluate them with its native evaluator. SE(3)-WAM reaches [XX.X]% SR against [XX.X]% for [the strongest baseline] (Table 2; Appendix C.3). As on Dynamic-14, it reads the simulator pose of the target.

4.3. Ablations and Analysis

Five controls each change one component of the final model and keep the data, recipe, and training budget, with three training seeds each (Table 3, Fig. 5). They test the argument of Sec. 3.2: the target history supplies the velocity on which e_v in Eq. (2) depends, future supervision shapes the representation without adding inference time, and the two geometric components and a uniform-fidelity forecast test the allocation of precision by role. We evaluate them with native latency and with 150 ms of added latency (per-seed results and a sensitivity analysis in Appendix C.4).

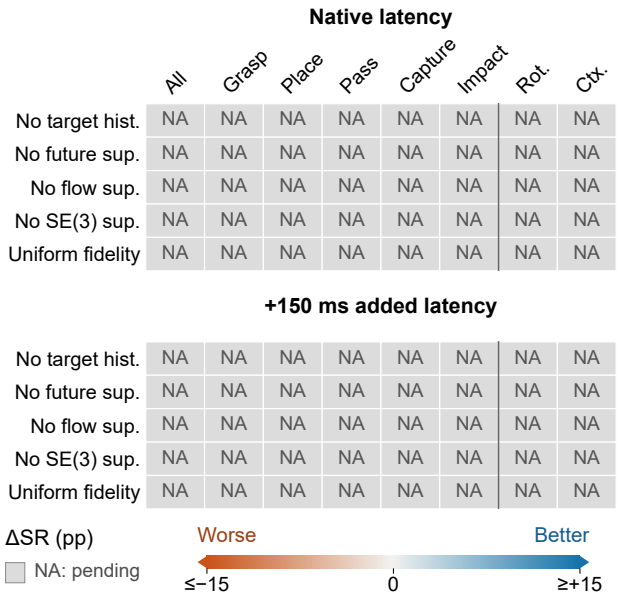


Figure 5. **Ablation effects by task (design preview; data pending).** ΔSR = SR(control) − SR(full), in percentage points; colors saturate at ±15. Controls follow Table 3. Rot./Ctx.: predeclared, overlapping rotation-sensitive/context-active groups (Table S3). Planned estimates average three seeds per scene and weight tasks equally, retaining `t3_phase`. Bold will mark unadjusted paired 95% scene-bootstrap intervals excluding zero, conditional on trained weights (Appendix B.4). Grey NA: no eligible paired results or estimates.

Table 3. **Core ablations** of the final 24-layer model on held-out Dynamic-14 scenes (three training seeds). Each row changes one component of SE(3)-WAM (*). Uniform-fidelity forecast: the SE(3) and scene-flow targets replaced by a forecast of the whole frame at one fidelity. Native, +150 ms: success (%) with native latency and with 150 ms of added latency. p95 lat.: p95 latency, the time-to-action (ms).

Variant	Native	+150 ms	p95 lat. (ms)
SE(3)-WAM *	—	—	—
No target history	—	—	—
No future supervision	—	—	—
No scene-flow supervision	—	—	—
No dedicated SE(3)	—	—	—
Uniform-fidelity forecast	—	—	—

Target history. The no-history control removes the past target poses with their velocity and acceleration features and keeps the current image, the current target pose, and the robot history; unlike the other controls, it changes the input. Without the history, SR drops by [X.X] points with native latency and by [X.X] points with 150 ms of added

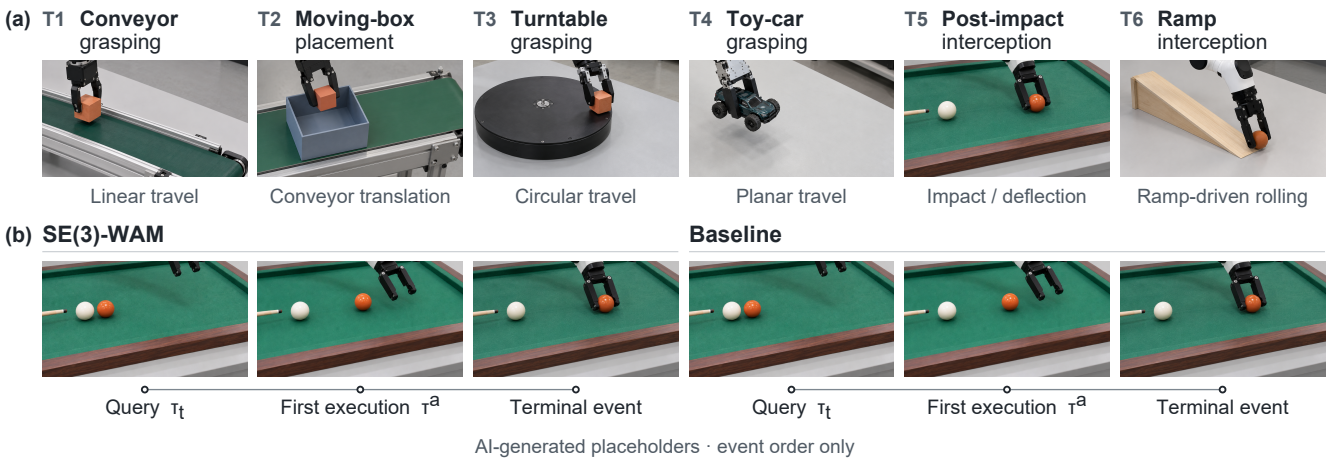


Figure 6. **Real-robot layout preview.** (a) Six task concepts; T6 shows a rolling ball. (b) T5 query capture τ_t, first execution τ^a, and terminal event. AI-generated placeholders repeat one storyboard for both methods; no recorded trial or measurement is shown.

Table 4. **Real-robot results.** Success rate (%) on the six tasks of Sec. 4.4, 20 trials per task and method; All pools the trials. p95 lat.: p95 latency, the end-to-end time-to-action (ms) on one RTX 5090, including tracking for SE(3)-WAM. DP: Diffusion Policy. Intervals in Appendix D.

Method	T1	T2	T3	T4	T5	T6	All	p95 lat.
$\pi_{0.5}$	—	—	—	—	—	—	—	—
DynamicWAM	—	—	—	—	—	—	—	—
DP	—	—	—	—	—	—	—	—
SE(3)-WAM	—	—	—	—	—	—	—	—

latency, as Eq. (2) predicts: a longer lead amplifies the velocity error that the history removes. Because the history relies on pose estimates, we also perturb them; under synthetic correlated pose noise, SE(3)-WAM keeps [XX.X]% SR (Appendix C.5).

Future supervision. The no-future control removes all forecast losses, the SE(3), scene-flow, and future appearance targets, and keeps the current-target RGB reconstruction. Its inference is identical to that of SE(3)-WAM, yet SR drops by [X.X] points with native latency and by [X.X] points with 150 ms of added latency, most on [tasks with non-uniform target motion]. Offline, the learned SE(3) forecast is also more accurate than constant-velocity and constant-acceleration extrapolation of the same history (Table S9).

Allocation of precision. Three controls keep all auxiliary targets and change the geometric forecast. Removing the scene-flow supervision lowers SR by [X.X] points; because the flow also covers the target and the robot, and the appearance targets remain, this measures the contribution of coarse scene-motion supervision rather than of all context information. Removing the dedicated SE(3) supervision leaves the target with only the coarse flow that every part of the scene receives and lowers SR by [X.X] points. If precision follows the roles, the SE(3) supervision should matter most on tasks with a tight pose tolerance or a rotation-sensitive contact, and the flow supervision on tasks whose context constrains or drives the target. [Whether Fig. 5 shows this pattern; if it does not, state that the controls establish the contribution of each component but not the allocation by role.] The uniform-fidelity control replaces the SE(3) and scene-flow targets by [future latents of a frozen video VAE] of the whole frame at the same horizons, so that every part of the scene is forecast at one fidelity; at the same inference cost, it trails SE(3)-WAM by [X.X] points. [One-sentence summary of which components matter, and where.]

4.4. Real-Robot Experiments

Setup. We evaluate six tasks on a Franka arm ([model]) with [gripper model] and a fixed RealSense D435 RGB-D camera (Fig. 6a; Appendix D): conveyor grasping (T1), placement into a moving receptacle (T2), turntable grasping (T3), moving toy-car grasping (T4), post-impact ball interception (T5), and ramp rolling/sliding interception (T6). Baselines are $\pi_{0.5}$ (VLA), DynamicWAM (dynamic WAM), and Diffusion Policy [12] (reactive). All methods train or fine-tune on the same 100 teleoperated demonstrations per task and run 20 trials per task with shared trial keys (Appendix D.3). Baselines receive RGB [and a wrist camera if their recipe uses one]; SE(3)-WAM also receives FoundationPose [44] target estimates, updated at every control-period boundary before its query (Appendix D.2). End-to-end latencies include tracking (Table 4).

Results. SE(3)-WAM succeeds in [XX.X]% of all trials, against [XX.X]% for [the strongest baseline], a paired difference of [X.X] points (95% interval [[X.X, X.X]]; Table 4). Its p95 latency, including tracking, is [XX] ms, [within one control period], whereas [name] needs [XX] ms. The margin is largest on [tasks], and the dominant failure of SE(3)-WAM is [failure mode] (Appendix D.4).

5. Conclusion

SE(3)-WAM spends precision where contact depends on it: a metric SE(3) history and forecast of the target and a sparse 3D flow of the whole scene, with the forecasts used only as training supervision. It reaches [XX.X]% success on Dynamic-14, [XX.X]% on DOMINO, and [XX]% on six real-robot tasks at a p95 latency of [XX] ms, improving on reactive, dynamic, and world–action policies. [The ablations show that the target history matters most under added latency and that future supervision improves control without changing the inference computation.] [State what the dedicated SE(3), scene-flow, and uniform-fidelity controls support: a better forecast design, or, if the task groups also agree, precision allocated by role.] Accounting for inference latency [state how success rates and relative performance differ from paused evaluation].

Limitations. SE(3)-WAM requires a specified rigid target and reliable estimates of its pose, and its forecast does not model how the robot's own contact changes the target's motion. Its target-trajectory head predicts a single future per horizon and does not represent uncertainty or multimodal motion, which an uncertainty-aware forecast could address.

LLM Usage Statement

We used OpenAI Codex to assist with literature review, code implementation and debugging, experiment execution, and language editing of the manuscript. Codex also assisted in developing plotting code and refining figure and manuscript layouts. The authors reviewed AI-assisted content, checked references against original sources, and validated the implementation through testing and experiments. The authors take full responsibility for the final manuscript, experimental results, and conclusions.

References

[1] Iretiayo Akinola, Jingxi Xu, Shuran Song, and Peter K. Allen. Dynamic grasping with reachability and motion awareness. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 9422–9429, 2021. 2

[2] Peter K. Allen, Aleksandar Timcenko, Billibon Yoshimi, and Paul Michelman. Automated tracking and grasping of a moving object with a robotic hand-eye system. *IEEE Transactions on Robotics and Automation*, 9(2):152–165, 1993. 2

[3] Yaakov Bar-Shalom, X. Rong Li, and Thiagalingam Kirubarajan. *Estimation with Applications to Tracking and Navigation: Theory, Algorithms and Software*. Wiley, 2001.

[4] Berthold Bäuml, Thomas Wimböck, and Gerd Hirzinger. Kinematically optimal catching a flying ball with a hand-arm-system. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2010. 2

[5] Homanga Bharadhwaj, Roozbeh Mottaghi, Abhinav Gupta, and Shubham Tulsiani. Track2Act: Predicting point tracks from internet videos enables generalizable robot manipulation. In *European Conference on Computer Vision*, 2024. 2

[6] Kevin Black, Noah Brown, James Darpinian, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. In *Proceedings of The 9th Conference on Robot Learning*, pages 17–40, 2025. 1, 6, 17

[7] Kevin Black, Noah Brown, Danny Driess, et al. π_0: A vision-language-action flow model for general robot control. In *Proceedings of Robotics: Science and Systems*, 2025. 1

[8] Kevin Black, Manuel Galliker, and Sergey Levine. Real-time execution of action chunking flow policies. In *Advances in Neural Information Processing Systems*, pages 33383–33407, 2025. 2, 5

[9] Arunkumar Byravan and Dieter Fox. SE3-Nets: Learning rigid body motion using deep neural networks. In *IEEE International Conference on Robotics and Automation*, pages 173–180, 2017. 2

[10] Arunkumar Byravan, Felix Leeb, Franziska Meier, and Dieter Fox. SE3-Pose-Nets: Structured deep dynamics models for visuomotor control. In *IEEE International Conference on Robotics and Automation*, 2018. 2

[11] Chongkei Chang and Zhidong Deng. DynaWM: A base-VLA-guided world foundation model for moving-object manipulation. *arXiv preprint arXiv:2607.02604*, 2026. 2

[12] Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin C. M. Burchfiel, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Proceedings of Robotics: Science and Systems*, 2023. 1, 6, 8, 17, 21

[13] David Emukpere, Romain Deffayet, Bingbing Wu, Romain Brégier, Michael Niemaz, Jean-Luc Meunier, Denys Proux, Jean-Michel Renders, and Seungsu Kim. Disentangled object-centric image representation for robotic manipulation. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 15873–15879, 2025. 3

[14] Heng Fang, Shangru Li, Shuhan Wang, Xuanyang Xi, Dingkang Liang, and Xiang Bai. Towards generalizable robotic manipulation in dynamic environments. In *European Conference on Computer Vision*, pages 314–333, 2026. 1, 2, 6, 7, 17, 18

[15] Gunnar Farnebäck. Two-frame motion estimation based on polynomial expansion. In *Proceedings of the Scandinavian Conference on Image Analysis*, pages 363–370, 2003. 24

[16] Jun Guo, Xiaojian Ma, Yikai Wang, Min Yang, Huaping Liu, and Qing Li. FlowDreamer: A RGB-D world model with flow-based motion representations for robot manipulation. *IEEE Robotics and Automation Letters*, 11(3):2466–2473, 2026. 2

[17] Kaiming He, Xinlei Chen, Saining Xie, Yanghao Li, Piotr Dollár, and Ross Girshick. Masked autoencoders are scalable vision learners. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 16000–16009, 2022. 3, 13, 26

[18] Cheng-Chun Hsu, Bowen Wen, Jie Xu, Yashraj Narang, Xiaolong Wang, Yuke Zhu, Joydeep Biswas, and Stan Birchfield. SPOT: SE(3) pose trajectory diffusion for object-centric manipulation. In *IEEE International Conference on Robotics and Automation*, 2025. 2

[19] Wenlong Huang, Yu-Wei Chao, Arsalan Mousavian, Ming-Yu Liu, Dieter Fox, Kaichun Mo, and Li Fei-Fei. PointWorld: Scaling 3D world models for in-the-wild robotic manipulation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 20765–20779, 2026. 2, 14

[20] Alexander Khazatsky, Karl Pertsch, Suraj Nair, et al. DROID: A large-scale in-the-wild robot manipulation dataset. In *Proceedings of Robotics: Science and Systems*, 2024. 6, 14

[21] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, et al. OpenVLA: An open-source vision-language-action model. In *Conference on Robot Learning*, pages 2679–2713, 2025. 1

[22] Shuang Li, Yihuai Gao, Dorsa Sadigh, and Shuran Song. Unified video action model. In *Proceedings of Robotics: Science and Systems*, 2025. 2, 5

[23] Haoran Liao, Pengyue Wang, Shuoyu Chen, Kehan Cheng, Xuhang Chen, Yuhao Lin, Mu Lin, Zhizhao Liang, Xiaoyi Fan, Chengyi Xing, Dan Niu, Yi-Lin Wei, and Wei-Shi Zheng. DynamicManip: Enabling dynamic manipulation from a single static demonstration. *arXiv preprint arXiv:2608.01452*, 2026. 2

[24] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matthew Le. Flow matching for generative modeling. In *International Conference on Learning Representations*, 2023. 3

[25] Shilong Liu, Zhaoyang Zeng, Tianhe Ren, Feng Li, Hao Zhang, Jie Yang, Qing Jiang, Chunyuan Li, Jianwei Yang,

Hang Su, Jun Zhu, and Lei Zhang. Grounding DINO: Marrying DINO with grounded pre-training for open-set object detection. In *European Conference on Computer Vision*, 2024. 21

[26] Yuejiang Liu, Jubayer Hamid, Annie Xie, Yoonho Lee, Max Du, and Chelsea Finn. Bidirectional decoding: Improving action chunking via guided test-time sampling. In *International Conference on Learning Representations*, pages 4594–4627, 2025. 2

[27] Yushan Liu, Peibo Sun, Shoujie Li, Yifan Xie, Lingfeng Zhang, Xintao Chao, Shiyuan Dong, Fang Chen, Xiao-Ping Zhang, and Wenbo Ding. OA-WAM: Object-addressable world action model for robust robot manipulation. *arXiv preprint arXiv:2605.06481*, 2026. 2

[28] Yunfan Lou, Hewen Gao, Xiyu Zhu, Zhuoran Qiao, Xuan Han, Yifan Yang, Yifan Ye, Boxian Yao, and Zhibo Pang. DynamicWAM: Dual-path motion conditioning for world-action models in dynamic manipulation. *arXiv preprint arXiv:2608.00793*, 2026. 1, 2, 6, 17, 24

[29] Siyuan Ma, Yutian Zhang, Boshi Zhang, Qinglian Wu, Jiaqi Zhai, Dong Wei, and Xiaojin Huang. ForeTime-VLA: Causal future-token distillation from a world action model for conveyor-belt manipulation. *arXiv preprint arXiv:2608.20735*, 2026. 2

[30] Lucas Manuelli, Wei Gao, Peter Florence, and Russ Tedrake. kPAM: Keypoint affordances for category-level robotic manipulation. In *Robotics Research: The 19th International Symposium ISRR*, pages 132–157. Springer, 2022. 2

[31] Naresh Marturi, Marek Kopicki, Alireza Rastegarpanah, et al. Dynamic grasp and trajectory planning for moving objects. *Autonomous Robots*, 43(5):1241–1256, 2019. 2

[32] Nikhila Ravi, Valentin Gabeur, Yuan-Ting Hu, Ronghang Hu, Chaitanya Ryali, Tengyu Ma, Haitham Khedr, Roman Rädle, Chloe Rolland, Laura Gustafson, Eric Mintun, Junting Pan, Kalyan Vasudev Alwala, Nicolas Carion, Chao-Yuan Wu, Ross Girshick, Piotr Dollár, and Christoph Feichtenhofer. SAM 2: Segment anything in images and videos. In *International Conference on Learning Representations*, pages 28085–28128, 2025. 21

[33] Mohit Shridhar, Lucas Manuelli, and Dieter Fox. Perceiver-actor: A multi-task transformer for robotic manipulation. In *Conference on Robot Learning*, pages 785–799, 2023. 2

[34] Mustafa Shukor, Dana Aubakirova, Francesco Capuano, Pepijn Kooijmans, Steven Palma, et al. SmolVLA: A vision-language-action model for affordable and efficient robotics. *arXiv preprint arXiv:2506.01844*, 2025. 2

[35] Oriane Siméoni, Huy V. Vo, Maximilian Seitzer, Federico Baldassarre, Maxime Oquab, Cijo Jose, Vasil Khalidov, Marc Szafraniec, Seungeun Yi, Michaël Ramamonjisoa, et al. DINOv3. *arXiv preprint arXiv:2508.10104*, 2025. 4, 14, 24, 26

[36] Otto J. M. Smith. Closer control of loops with dead time. *Chemical Engineering Progress*, 53(5):217–219, 1957. 2

[37] Rustin Soraki, Homanga Bharadhwaj, Ali Farhadi, and Roozbeh Mottaghi. ObjectForesight: Predicting future 3D object trajectories from human videos. In *European Conference on Computer Vision*, pages 437–453, 2026. 2

[38] Yue Su, Sijin Chen, Haixin Shi, Mingyu Liu, Zhengshen Zhang, Ningyuan Huang, Weiheng Zhong, Zhengbang Zhu, Yuxiao Liu, and Xihui Liu. World guidance: World modeling in condition space for action generation. In *Proceedings of the 43rd International Conference on Machine Learning*, pages 115917–115933, 2026. 2

[39] Shahram Najam Syed, Arthur Jakobsson, Haoran Hao, and Jeffrey Ichnowski. Intercepting the future: Latent-space predictive world model for dynamic VLA manipulation. In *Conference on Robot Learning*, 2026. Accepted for publication. 1, 2

[40] Jiaming Tang, Yufei Sun, Yilong Zhao, Shang Yang, Yujun Lin, Zhuoyang Zhang, James Hou, Yao Lu, Zhijian Liu, and Song Han. VLASH: Real-time VLAs via future-state-aware asynchronous inference. *arXiv preprint arXiv:2512.01031*, 2025. 2

[41] Yang Tian, Sizhe Yang, Jia Zeng, Ping Wang, Dahua Lin, Hao Dong, and Jiangmiao Pang. Predictive inverse dynamics models are scalable learners for robotic manipulation. In *International Conference on Learning Representations*, pages 92033–92052, 2025. 2

[42] Emanuel Todorov, Tom Erez, and Yuval Tassa. MuJoCo: A physics engine for model-based control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 5026–5033, 2012. 5, 15

[43] Yuran Wang, Siqiao Huang, Mingleyang Li, Chenhao Zhang, Jiaqi Liang, Weiyang Jin, Yue Chen, Xuemin Chi, Donghao Zhou, Qize Yu, Yu-Kai Wang, Yuhan Rui, Shenzhe Yao, Zhen Yuan, Zhenhao Shen, Kefei Zhu, Zijie Zhu, Ning Gao, Xiaowei Chi, Guanqi He, Shanghang Zhang, Hao Dong, Lin Shao, and Hang Zhao. OpenWAM: An open, modular exploration towards systematic world-action model pretraining. *arXiv preprint arXiv:2609.07398*, 2026. 1, 2, 6, 17

[44] Bowen Wen, Wei Yang, Jan Kautz, and Stan Birchfield. FoundationPose: Unified 6D pose estimation and tracking of novel objects. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 17868–17879, 2024. 8, 21

[45] Chuan Wen, Xingyu Lin, John Ian Reyes So, Kai Chen, Qi Dou, Yang Gao, and Pieter Abbeel. Any-point trajectory modeling for policy learning. In *Proceedings of Robotics: Science and Systems*, 2024. 2

[46] Hongtao Wu, Ya Jing, Chilam Cheang, Guangzeng Chen, Jiafeng Xu, Xinghang Li, Minghuan Liu, Hang Li, and Tao Kong. Unleashing large-scale video generative pre-training for visual robot manipulation. In *International Conference on Learning Representations*, pages 10641–10662, 2024. 2

[47] Haozhe Xie, Beichen Wen, Jiarui Zheng, Zhaoxi Chen, Fangzhou Hong, Haiwen Diao, and Ziwei Liu. DynamicVLA: A vision-language-action model for dynamic object manipulation. In *Advances in Neural Information Processing Systems*, 2026. Accepted for publication. 2, 6, 17

[48] Mengda Xu, Zhenjia Xu, Yinghao Xu, Cheng Chi, Gordon Wetzstein, Manuela Veloso, and Shuran Song. Flow as the cross-domain manipulation interface. In *Conference on Robot Learning*, pages 2475–2499, 2025. 2

[49] Seonghyeon Ye, Yunhao Ge, Kaiyuan Zheng, Shenyuan Gao, Sihyun Yu, George Kurian, Suneel Indupuru, et al.

World action models are zero-shot policies. *arXiv preprint arXiv:2602.15922*, 2026. 1, 2

[50] Hanyang Yu, Haitao Lin, Jingbo Zhang, Wenyao Zhang, Chenghao Gu, Heng Li, and Ping Tan. MaskWAM: Unifying mask prompting and prediction for world-action models. *arXiv preprint arXiv:2606.13515*, 2026. 2

[51] Chengbo Yuan, Chuan Wen, Tong Zhang, and Yang Gao. General flow as foundation affordance for scalable robot learning. In *Conference on Robot Learning*, pages 1541–1566, 2025. 2

[52] Shanglin Yuan, Weiheng Zhao, Xin Shi, Haoyi Jiang, Xianda Guo, Liu Liu, Wenyu Liu, Wei Sui, and Xinggang Wang. DreamWAM: Beyond RGB future prediction for world action models. *arXiv preprint arXiv:2608.04996*, 2026. 2

[53] Tianyuan Yuan, Zibin Dong, Yicheng Liu, and Hang Zhao. Fast-WAM: Do world action models need test-time future imagination? In *Advances in Neural Information Processing Systems*, 2026. Accepted for publication. 2, 5

[54] Peng Yun, Shouwang Huang, Hao Li, Jinxi Li, Jianan Wang, and Bo Yang. PhysMani: Physics-principled 3D world model for dynamic object manipulation. In *European Conference on Computer Vision*, pages 20–39, 2026. 2

[55] Yanjie Ze, Gu Zhang, Kangning Zhang, Chenyuan Hu, Muhan Wang, and Huazhe Xu. 3D diffusion policy: Generalizable visuomotor policy learning via simple 3D representations. In *Proceedings of Robotics: Science and Systems*, 2024. 2

[56] Wenyao Zhang, Hongsi Liu, Zekun Qi, Yunnan Wang, XinQiang Yu, Jiazhao Zhang, Runpei Dong, Jiawei He, He Wang, Zhizheng Zhang, Li Yi, Wenjun Zeng, and Xin Jin. DreamVLA: A vision-language-action model dreamed with comprehensive world knowledge. In *Advances in Neural Information Processing Systems*, pages 24195–24228, 2025. 1, 2, 6, 17

[57] Yifan Zhang, Ruiping Wang, and Xilin Chen. Dynamic behavior cloning with temporal feature prediction: Enhancing robotic arm manipulation in moving object tasks. *IEEE Robotics and Automation Letters*, 10(6):5209–5216, 2025. 2

[58] Ruiteng Zhao, Zhengshen Zhang, Yue Su, Wenshuo Wang, Jiahui Li, Zhiyuan Yang, Francis E. H. Tay, Marcelo H. Ang, and Haiyue Zhu. SG-WAM: Self-guided world modeling in geometry-aware policy space. *arXiv preprint arXiv:2608.01397*, 2026. 2

[59] Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In *Proceedings of Robotics: Science and Systems*, 2023. 6, 17

[60] Ruijie Zheng, Jing Wang, Scott Reed, et al. FLARE: Robot learning with implicit world modeling. In *Proceedings of The 9th Conference on Robot Learning*, pages 3952–3971. PMLR, 2025. 2

[61] Chuning Zhu, Raymond Yu, Siyuan Feng, Benjamin Burchfiel, Paarth Shah, and Abhishek Gupta. Unified world models: Coupling video and action diffusion for pretraining on large robotic datasets. In *Proceedings of Robotics: Science and Systems*, 2025. 2

[62] Yuke Zhu, Josiah Wong, Ajay Mandlekar, Roberto Martín-Martín, Abhishek Joshi, Kevin Lin, Abhiram Maddukuri, Soroush Nasiriany, and Yifeng Zhu. robosuite: A modular simulation framework and benchmark for robot learning. *arXiv preprint arXiv:2009.12293*, 2020. 5, 15

[63] Brianna Zitkovich, Tianhe Yu, Sichun Xu, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning*, pages 2165–2183, 2023. 1

A. Method and Training Details

A.1. Error Decomposition

This subsection derives Eq. 2 and the rotation and context terms behind the task groups of Appendix B.1, and shows how the error of an estimate scales with the lead and the motion.

Translation. Write the true target position at the execution time as $\mathbf{p}(\tau^{\mathrm{a}}) = \mathbf{p}(\tau_t) + \mathbf{v}\Delta + \boldsymbol{\rho}$, with the velocity $\mathbf{v}$ at τ_t and the remaining motion $\boldsymbol{\rho}$, and the policy's estimate as $\hat{\mathbf{p}}(\tau^{\mathrm{a}}) = \hat{\mathbf{p}}(\tau_t) + \hat{\mathbf{v}}\hat{\Delta} + \hat{\boldsymbol{\rho}}$. Subtracting and adding $\mathbf{v}\hat{\Delta}$ gives

$$\begin{aligned}\hat{\mathbf{p}}(\tau^{\mathrm{a}}) - \mathbf{p}(\tau^{\mathrm{a}}) = \big[\hat{\mathbf{p}}(\tau_t) - \mathbf{p}(\tau_t)\big] + (\hat{\mathbf{v}} - \mathbf{v})\hat{\Delta} \\ + \mathbf{v}(\hat{\Delta} - \Delta) + (\hat{\boldsymbol{\rho}} - \boldsymbol{\rho}),\end{aligned} \tag{S1}$$

and the triangle inequality yields Eq. 2 with $e_v = \|\hat{\mathbf{v}} - \mathbf{v}\|$ and $e_{\mathrm{dyn}} = \|\hat{\boldsymbol{\rho}} - \boldsymbol{\rho}\|$. The bound holds for any split of the estimate into $\hat{\mathbf{v}}\hat{\Delta}$ and $\hat{\boldsymbol{\rho}}$, and e_{dyn} absorbs the higher-order part of the timing error. The residual $\boldsymbol{\rho}$ is the motion that constant-velocity extrapolation misses. Under a constant acceleration $\mathbf{a}$, $\boldsymbol{\rho} = \frac{1}{2}\mathbf{a}\Delta^2$, which an acceleration estimate from the history reduces; impacts, rolling contact, and friction changes leave a remainder that only a model of the dynamics can reduce.

Rotation and the contact point. Write the true orientation as $R(\tau^{\mathrm{a}}) = \mathrm{Exp}(\boldsymbol{\omega}\Delta + \boldsymbol{\eta})R(\tau_t)$, with the angular velocity $\boldsymbol{\omega}$ and the remaining rotation $\boldsymbol{\eta}$, and its estimate likewise. The rotation angle is bi-invariant, and Exp is 1-Lipschitz from $\mathbb{R}^3$ to SO(3), so for general rotations

$$\begin{aligned}\theta &= \big\| \mathrm{Log}(\hat{R}(\tau^{\mathrm{a}})R(\tau^{\mathrm{a}})^{\top})^{\vee} \big\| \\ &\le \theta_{\mathrm{loc}} + \|\hat{\boldsymbol{\omega}}\hat{\Delta} + \hat{\boldsymbol{\eta}} - \boldsymbol{\omega}\Delta - \boldsymbol{\eta}\| \\ &\le \theta_{\mathrm{loc}} + \hat{\Delta}\, e_\omega + \|\boldsymbol{\omega}\|\, e_\tau + e_\eta,\end{aligned} \tag{S2}$$

with the current orientation error θ_{loc}, $e_\omega = \|\hat{\boldsymbol{\omega}} - \boldsymbol{\omega}\|$, and $e_\eta = \|\hat{\boldsymbol{\eta}} - \boldsymbol{\eta}\|$. A contact point at $\mathbf{c}$ in the object frame, with $\|\mathbf{c}\| = r$, is off by $(\hat{\mathbf{p}} - \mathbf{p}) + (\hat{R}R^{\top} - I)R\mathbf{c}$ at τ^{a}, and the operator norm of $\hat{R}R^{\top} - I$ is $2\sin(\theta/2)$, so its error is

$$e_{\mathrm{c}} \le e + 2r\sin(\theta/2) \le e + r\theta, \tag{S3}$$

with $e = \|\hat{\mathbf{p}}(\tau^{\mathrm{a}}) - \mathbf{p}(\tau^{\mathrm{a}})\|$. A grasp or placement constrains several points spread over the body, such as the two finger contacts or the arms of a T-shaped part, so no single tracked point removes the term $r\theta$: the orientation error enters the contact scaled by the lever arm, and the target needs its full SE(3) state.

Context. Let m be the largest margin that a feasible path can keep between the robot and the carried object and the context it passes, measured to the predicted positions of the context at τ^{a}. If the executed path keeps this margin, a collision requires a context error of at least m, so a coarse estimate of the surroundings suffices when $m \gg \varepsilon$; the no-scene-flow control of Sec. 4.3 tests this coarse supervision directly. A driver, such as a conveyor, a ramp, or a striker, acts on the contact through the motion it imparts to the target, which enters Eq. 2 through the target's own terms. The SE(3) history and forecast represent this motion precisely, so the driver needs no precise state of its own.

Table S1. Architecture of SE(3)-WAM. Heads are skipped at deployment.

Module	Configuration
Visual encoder	frozen MAE ViT-B/16, 224×224 input, 196 patch tokens
Target-state tokenizer	$28 \to 1024 \to 1024$ GELU MLP, additive relative-time embedding
Robot-state tokenizer	linear, $7 \to 1024$
Tokens	212 scene (196 patch, 8 target-state, 8 robot), 16 summary, 8 action
Backbone	24 layers, width 1024, 16 heads, shared attention, SiLU FFN 4096 per stream, 504.0M parameters
Pose head	linear, mean summary state $\to 8 \times 6$
Flow head	linear per patch token, $8 \times (3{+}1)$
Appearance heads	linear per patch token: depth and moving-surface RGB as 16×16 pixels, 1024-d DINOv3 ViT-L/16 features
Target reconstruction	$1024 \to 256 \to 3072$ GELU MLP with sigmoid, 32×32 RGB
Action stream	flow matching, adaptive layer norm on the flow time, 5 Euler steps

A.2. Architecture

Table S1 lists the configuration of every module, in the order of the data flow described below.

Scene tokens. The frozen MAE ViT-B/16 [17] encodes the agent-view RGB into 196 patch tokens; we selected it on development scenes (Appendix E.4). Each patch token adds a learned view embedding, shared by all patches in our single-view setting, and linear embeddings of its mean valid metric depth and of the base-frame position of its back-projected center pixel. These positions also drive 3D rotary embeddings of the patch queries and keys, which no other token receives, and the same center points are the source points of the sparse 3D flow. Past images never enter the model.

State tokens. The 8 target-state tokens sample the offsets $-350, -300, \ldots, 0$ ms. Each is a 25-dimensional descrip-

tor $(\mathbf{p}, \log R, \Delta\mathbf{p}, \log \Delta R, \Delta t, \mathbf{v}, \boldsymbol{\omega}, \mathbf{a}, \boldsymbol{\alpha})$ with dimensions $(3, 3, 3, 3, 1, 3, 3, 3, 3)$: slots 0–6 describe the seven intervals between consecutive offsets, with velocities from interval displacements over the elapsed time and accelerations from adjacent velocities over the spacing of the interval centers, and slot 7 holds the current absolute pose in the base frame. We standardize each dimension with training statistics, zero invalid dimensions, and append three validity flags (pose, interval, and acceleration); a slot without any valid feature is masked out of attention. On the real robot, the tracker updates the pose at every control-period boundary (Appendix D.2), so the history keeps the same 50 ms spacing. The robot tokens hold the end-effector position, rotation vector, and measured gripper opening at the same offsets, and 16 learned embeddings form the summary tokens. The policy has no language token.

Backbone and mask. Each layer applies one attention over all tokens with query, key, value, and output projections shared by both streams, followed by separate SiLU feed-forward networks for the world and the action tokens. The mask of Sec. 3.3 gives a blocked edge the logit $-\infty$ in every layer and head, in training and at deployment, so the world states are computed once per query and reused across the Euler steps.

Forecast and auxiliary heads. All heads are readouts of the final world states, trained jointly with the action stream and skipped at deployment; no token reads their outputs. The pose head reads the mean summary state and is trained with $\mathcal{L}_P$ (Eq. 4). The flow head gives each patch point's displacement and visibility logit at every horizon, trained with $\mathcal{L}_F$ (Eq. 5) and the scale $s = 0.7154605$ m; the visibility loss has weight 1 in simulation, and points that are valid but invisible still supervise their displacement. The appearance heads predict at every horizon the metric depth with a squared error on valid pixels, the RGB of surfaces that move by more than 1 mm, including the robot, with a squared error, and the DINOv3 features [35] with a squared error after channel-wise layer normalization. The target-reconstruction head recovers the current target's RGB region from the mean summary state with an absolute error. These four losses form $\mathcal{L}_{\text{aux}}$.

Action stream. Each of the 8 action tokens embeds a seven-dimensional command, a base-frame translation increment, a left-relative rotation vector, and a gripper command; the increments refer to the measured end-effector pose at application. At deployment, we integrate 5 Euler steps, reverse the normalization, decode the rotation vectors, and clip each axis to 0.05 m and 0.5 rad; the gripper follows the sign of its command. Appendix B.3 specifies how the chunks are executed.

A.3. Training on Dynamic-14

Each control step of a demonstration anchors one window of 8 query positions with their own causal histories and no attention across positions, which gives 1,324,374 windows from the 11,200 demonstrations; 350 further episodes form the validation split. For the flow labels, each patch-center pixel is back-projected with the rendered depth and attached to the local frame of the MuJoCo geometry rendered at it; the ground-truth rigid transforms of that geometry carry the same surface point to the future horizons, which gives its cumulative displacement in the base frame. A future point is visible if it lies in front of the camera and inside the image and its projected pixel renders the same geometry at a depth within $\max(1\,\text{mm},\ 0.001z)$ of the point's depth z. Command j of the chunk for query t is labeled with the demonstration command issued at $\tau_t + j\Delta_a$, since d_{task} applies only at execution (Eq. 1), and commands are normalized with the mean and standard deviation of the valid commands of the training split. Each query position draws 8 noise–time pairs for its chunk and averages their losses, which reduces the variance of the action gradient without enlarging the batch, and each loss is normalized by its count of valid labels in the global batch. The DINOv3 features of future frames are cached once per episode. Table S2 lists the recipe.

Table S2. Training hyperparameters on Dynamic-14.

Item	Value
Optimizer	AdamW, $\beta = (0.9, 0.999)$, $\epsilon = 10^{-8}$, weight decay 10^{-4}
Learning rate	2×10^{-4}, 5,174 warm-up updates from 0, cosine decay to 0
Batch, updates	global batch 512 windows, 20 epochs (51,740 updates)
Loss weights	$\lambda_P = \lambda_F = 1$, $\lambda_R = 0.1$, $\lambda_{\text{aux}} = 0.1$ per target
Action noise	8 noise–time pairs per query position
Precision	trainable weights FP32, frozen encoder BF16, gradient clipping at 0.1
Inference	FP32, world states cached within a query
Hardware	[N] RTX PRO 6000 GPUs, [X] hours per run

A.4. Pretraining on DROID and Fine-Tuning

Data. The real-robot base trains on DROID [20], recorded at 15 Hz, with the 3D point tracks and camera calibrations that PointWorld released for it [19], matched to the raw episodes by exact agreement of robot states, commands, and camera identity. It uses the first exterior camera with its depth. Of 42,935 matched episodes, 42,087 yield 598,889 anchor windows, 538,740 from 37,874 training and 60,149 from 4,213 validation episodes. Each anchor expands into 8 consecutive query positions; the anchor carries the flow

and DINOv3 labels, and the others carry action labels only. All times are capture timestamps, and the flow and DINOv3 targets keep PointWorld's record times. We keep the 96.88% of action slots with an issued command, without a filter on speed, so that the base is not biased toward slow motion.

Objectives. DROID provides no object poses, so the base has no target-state tokens or pose head. It reads the current RGB-D frame and the robot history (204 scene tokens) and trains the action loss and the sparse 3D flow with weight 1 and the future DINOv3 features with weight 0.1; PointWorld records only visible positions, so the visibility loss is off. Most DROID objects move only when the robot moves them, so the base mainly learns metric geometry and the motion caused by the robot's own actions; the exogenous motion of our tasks and the target pose enter during fine-tuning.

Flow labels. Each patch of the 14×14 grid follows one PointWorld track: among the tracks that start inside the patch, are initially visible, and have a valid depth, the one with the largest image motion consistent with its 3D displacement, and otherwise the one nearest to the patch center, so that moving objects missed by a fixed center point are covered. The label is the track's displacement from its start; 47.05% of the point–horizon slots carry a label.

Recipe and fine-tuning. The base has 48 layers at width 1024 (1.02B trainable parameters) and trains for 20 epochs (21,060 updates, 2,106 warm-up updates) at a global batch of 512 on 64 GPUs, with the optimizer and precision of Table S2. Each real-robot policy starts from the base and is fine-tuned on the demonstrations of its task [updates, learning rate, and batch size]; the target-state tokens and the heads that the base lacks are freshly initialized. No real-robot policy uses simulation weights, and the base sees no simulation data.

B. Dynamic-14 and Evaluation Protocol

B.1. Benchmark Design

Dynamic-14 comprises fourteen tasks in five families (Fig. 4; Sec. 4.1), which together cover translation, rotation, temporary occlusion, constrained passage, and abrupt changes of the target's motion. We will release the tasks, the accepted scenarios, the success checks, and the demonstrations.

Environment. The tasks run in MuJoCo 3.11.0 and robosuite 1.5.2 [42, 62] with a Franka Panda arm and a parallel gripper, driven by robosuite's operational-space pose controller at a control period of 50 ms and a physics step of 2 ms. A fixed third-person agent-view camera at $(0.68, -0.58, 1.62)$ m with a 40° vertical field of view renders 224×224 RGB-D images. An action is an end-effector increment in the robot base frame, a translation and a rotation vector, together with a gripper command; the shared action adapter clips each command to 0.10 m per translation axis and 0.5 rad per rotation-vector component.

Success and termination. Table S3 summarizes the success condition of each task. We check them at every physics step, an interrupted continuous condition resets its counter, and an episode ends at its first terminal event: success, a failure such as unsafe contact, leaving the workspace, or a drop, or the time limit.

Roles and task groups. Table S3 names the target of each task. A moving receptacle is the target, since its pose decides containment; in the passage tasks, the wire entry, which the wand must match within 30 mm, is the target, and the rest of the wire is context that no contact may touch. The two task groups of Fig. 5 were fixed before the ablations from the terms of Appendix A.1: a task is rotation-sensitive ($r\theta$) if its target rotates during the lead and its orientation matters at contact, which excludes the balls, and context-active if it has an obstacle clearance m or a driver (a conveyor, a ramp, or a striker).

B.2. Scenarios, Demonstrations, and Splits

Scenarios. Each scene samples the target's initial state and motion parameters uniformly within task-specific ranges, and discrete choices with equal probability: besides the speeds of Table S3, harmonic amplitudes of up to 35 mm at 0.25–0.7 Hz, yaw rates of up to 1.2 rad/s, wire rotation rates of 2.5–7.5°/s, and ramp slopes of 5–15° with friction set per object. An occluded scene is accepted only if the target is visible for at least eight initial frames, fully occluded for 0.5–3.0 s, and visible again for at least three frames. We keep the scenes that pass physical-feasibility checks and replay the stored accepted scenes at evaluation.

Demonstrations. The expert is a task-specific closed-loop state machine with privileged simulator state: the target's current pose and linear and angular velocity, the robot state, the task geometry, and true contact, grasp, and stage events. At every control period it, for example, extrapolates the target over a short horizon and tracks, descends, closes, and lifts in the grasping tasks, predicts the interception point of a rolling or flying object under gravity in the capture tasks, and follows the known wire path with phase compensation from the current angle and angular velocity in the passage tasks. Its commands are clipped to 0.05 m and 0.5 rad per axis. A demonstration is accepted if its scene is feasible, the expert succeeds, an independent replay of the recorded actions reproduces the episode, and its path, action changes, and jerk pass numerical quality checks. The training pool of 11,200 demonstrations, 800 per task, is drawn from the accepted

Table S3. **The fourteen Dynamic-14 tasks.** Target: the body whose pose the policy receives (Sec. 3.1). Speed: target speed range (capture tasks: at the ramp exit). Every task requires its stages in order without a terminal failure; a grasp requires bilateral fingerpad contact, a lift of at least 0.05 m, and a 0.5 s hold. Limit: episode time limit. Groups of Fig. 5: R, rotation-sensitive; C, context-active (Appendix B.1). The released task configurations contain the exact thresholds and scenario ranges.

Task (code)	Target	Speed (m/s)	Success	Limit (s)	Group
Harmonic grasp (grasp)	block	0.03–0.20	Grasp the block moving harmonically and yawing, lift, and hold.	8	R
Conveyor grasp (t1_belt)	part	0.04–0.10	Grasp the part on the conveyor, lift, and hold.	8	C
Translating grasp (t1_xyz)	part	0.04–0.15	Grasp the translating part, lift, and hold.	8	–
Rotating grasp (t1_so3)	part	0.04–0.15	Grasp the translating and rotating part ($\|\omega\| \leq 1.2$ rad/s), lift, and hold.	8	R
Occluded grasp (t1_occ)	part	0.04–0.10	Grasp the part on the conveyor, which a canopy hides for 0.5–3.0 s, without unsafe contact with the canopy; lift and hold.	8	C
Moving receptacle (t2_trans)	receptacle	0.03–0.06	Place the held object fully inside the moving receptacle and release it to rest.	10	–
Rotating receptacle (t2_se3)	T-shaped receptacle	0.03–0.06	Place the held T-shaped part fully inside the translating and rotating T-shaped receptacle and release it to rest.	10	R
Wire passage (t3_phase)	wire entry	[X]	Align the held wand's loop with the rotating wire within 30 mm, then pass it along the wire to the end without contact.	21	R, C
Full wire passage (t3_full)	wire entry	[X]	Grasp the wand, complete the passage, exit the wire, and release the wand.	32	R, C
Ball capture (t4_sphere)	ball	[0.4–0.7]	Catch the ball after it leaves the ramp and before it touches the tray, lift, and hold.	6	C
Ball capture on table (t4_sphere_tabletop)	ball	[0.4–0.7]	Grasp the ball while it still rolls on the table after the ramp, lift, and hold.	6	C
Slider capture (t4_slider)	slider	[0.4–0.8]	Grasp the block while it still slides on the table after the ramp, lift, and hold.	6	C
Can capture (t4_can)	can	[0.4–0.7]	Grasp the can while it still rolls on the table after the ramp, before it tips over, lift, and hold.	6	R, C
Post-impact grasp (t5_replan)	part	0.12–0.14	After a striker deflects the part on the conveyor, grasp it along its grasp axis, lift, and hold.	6	R, C

ones. We record them at 20 Hz with camera images, depth, robot states, issued commands, and simulator ground truth for target poses and scene geometry; the learned policies receive only the inputs of Sec. 3.

Splits. We select configurations on 1,400 development scenes (100 per task). All Dynamic-14 results use a separate held-out set of 1,400 scenes, generated with the same ranges and new source seeds that appear in no training, validation, or development episode; the first 100 feasible scenes in increasing seed order form each task's set, which all methods share.

B.3. Real-Time Evaluation

Executor. All methods share the executor of Sec. 3.4 on the 50 ms control clock; the total latency L_t of a query is its inference latency under the protocol plus any added latency. Four rules complete Sec. 3.4. A method's chunk holds its native number H_m of commands (Table S4), and a chunk with $k_t \geq H_m$ has expired entirely and is dropped. With $L_t = 0$, the next query starts one control period after capture. Before the first chunk of an episode arrives, and whenever a chunk runs out, the controller holds the current arm target and gripper command while physics advances; no command is repeated. The post-impact grasp applies each command 50 ms after it is issued ($d_{\text{task}} = 50$ ms in Eq. 1); all other tasks have $d_{\text{task}} = 0$.

Protocols. Under the real-time protocol, physics advances during each query by the method's native latency, drawn per query from its latency distribution measured on one RTX 5090. With paused physics, the simulation waits for inference; with equal latency, every method is charged the p95 latency of SE(3)-WAM; with original executors, each baseline keeps the execution scheme of its paper, such as temporal aggregation for ACT or streaming for DynamicVLA, at its native latency. The added-latency sweep of Table S7 adds 0, 50, 100, 150, or 200 ms after inference under the real-time protocol, and physics advances during it. The native-latency rollouts of ACT and Diffusion Policy ran on RTX 5090 and all other rollouts on RTX PRO 6000; every rollout replays the latency measured on RTX 5090, so the rollout hardware does not affect the results.

Latency measurement. We time all methods on one RTX 5090 (32 GB), the GPU of our robot, with exclusive use and batch size 1. Each method runs three rounds of thirty warm-up and 300 measured calls over three observation sequences of eight frames, with CUDA synchronization at call boundaries and a rotated method order, in the precision of its release; SE(3)-WAM uses FP32. A timed query starts from the newest observation, with the history already maintained, and ends when its first command is available. It includes all computation that needs the newest frame: preprocessing, image encoding, the online flow updates and motion descriptors of PUMA and DynamicWAM, sampling, and action decoding, and on the robot also tracking. For SE(3)-WAM, we time the forecast heads separately, although deployment skips them. This time-to-action is reported without rounding; the handover time $\ell_t = k_t\Delta_a$ rounds it up to the control period, and the action age of a command is its execution time minus the capture time. A query misses its deadline when its time-to-action exceeds 50 ms. Appendix C.2 reports the measured values.

B.4. Metrics and Statistics

The task-balanced success rate is

$$\mathrm{SR}(m) = \frac{100}{14}\sum_{k=1}^{14}\frac{1}{S_m n_k}\sum_{s=1}^{S_m}\sum_{i=1}^{n_k} y_{mski}, \qquad \text{(S4)}$$

for binary outcomes y_{mski} of method m, trained with seed s, on scene i of task k, with $S_m = 3$ training seeds for our variants and $S_m = 1$ for the baselines. The *position error at first contact* e_p is the distance of the end effector from the expert's contact frame on the target. In the two placement tasks, which start with the object held, the contact is that of the held object with the receptacle, and the contact frame is the end-effector pose of the expert's placement. e_p averages over the 13 tasks with a contact, excluding `t3_phase`, and over the episodes that make contact; failures without contact stay in the SR denominator.

Training seeds and intervals. The full model and the five ablation controls use training seeds 0, 1, and 2, and each baseline uses seed 0. We report each training seed and the cross-seed mean and standard deviation (Appendix C.4), and for the ablations also the comparison without the phase-only wire passage (`t3_phase`). For paired 95% intervals, we average the seeds per scene and bootstrap held-out scenes within each task, jointly across methods. These intervals quantify scene uncertainty for the trained weights, and we treat differences whose interval lies within ± 2 points as ties.

B.5. Baselines and Ablation Controls

Common conditions. Each baseline keeps its official architecture, chunk length, initialization, and training recipe (Table S4), trains once with seed 0 on the same 11,200 demonstrations, whose commands we map to its action space through the same Cartesian controller, and is configured on development scenes. All methods share the agent-view camera and receive the further inputs of Table 1; methods whose recipes expect wrist views receive masked or black wrist inputs.

Table S4. **Baseline training.** All methods train on the same 11,200 demonstrations with the official recipe of each method and run under the shared executor of Appendix B.3. Updates × batch: training budget. H: commands per returned chunk. Params: trainable parameters, frozen encoders excluded. DOM ckpt.: the released DynamicVLA checkpoint trained on DOM. DreamVLA samples with ten DDIM steps and classifier-free guidance of 1.5 and uses the better of its 40-epoch and a 20-epoch cosine schedule on development scenes; OpenWAM-α generates video and actions jointly with video decoding disabled.

Method	Initialization	Updates×batch	H	Params
ACT [59]	scratch	50,000 × 32	100	[X]
DP [12]	scratch	50,000 × 32	8	[X]
$\pi_{0.5}$ [6]	`pi05_base`	30,000 × 256	10	[X]
$\pi_{0.5}$+SE(3)	`pi05_base`	30,000 × 256	10	[X]
DynamicVLA [47]	DOM ckpt.	50,000 × 44	20	[X]
PUMA [14]	official	[X]×[X]	[X]	[X]
DreamVLA [56]	released	48,220 × 512	3	[X]
OpenWAM-α [43]	official	34,490 × 192	32	[X]
DynamicWAM [28]	official	40,000 × 128	16	[X]
DynamicWAM+SE(3)	official	40,000 × 128	16	[X]
SE(3)-WAM	scratch	51,740 × 512	8	523.8M

SE(3)-augmented baselines. $\pi_{0.5}$+SE(3) and DynamicWAM+SE(3) receive the eight target-state tokens of Appendix A.2 from the same pose source, with the same validity flags and masking. A two-layer MLP maps the tokens to the model's width. For $\pi_{0.5}$, we append them to the prefix after the image tokens, where the action expert attends to them; for DynamicWAM, they join its kinematic descriptors. The output layer of the MLP is zero-initialized, and training follows the base model's recipe.

Ablation controls. The five controls of Sec. 4.3 change one component each of the configuration of Table S2 and keep the data, the recipe, the update budget, the executor, and the training seeds 0, 1, and 2. *No target history* removes slots 0–6 of the target-state tokens, which carry the past poses with their velocity and acceleration features, and keeps slot 7 with the current pose (Appendix A.2). *No future supervision* removes $\mathcal{L}_P$, $\mathcal{L}_F$, and the three future appearance losses with their heads, and keeps the reconstruction of the current target region. *No scene-flow supervision* removes $\mathcal{L}_F$ and the flow readout, and *no dedicated SE(3)* removes $\mathcal{L}_P$ and the pose head; both keep the target-state tokens and all appearance

targets. The *uniform-fidelity forecast* replaces $\mathcal{L}_P$ and $\mathcal{L}_F$ by [a forecast of future latents of a frozen video VAE of the whole frame at the same horizons, its readout, and its loss weight]. Since every forecast head is skipped at deployment, only the no-history control changes the inputs at inference.

C. Additional Simulation Results

C.1. Per-Task Results

Table S5 breaks the held-out Dynamic-14 results of Table 1 down by task for every method, with the paired interval of SE(3)-WAM against the strongest other method on each task; its last two rows give the position error at first contact e_p and the number of episodes with a contact (Appendix B.4). With its native 100-step DDPM sampler, every chunk of Diffusion Policy has expired when it arrives, so none of its commands is executed. [Tasks on which every method fails or a baseline outperforms SE(3)-WAM, and the dominant failure of SE(3)-WAM.]

C.2. Evaluation Protocols and Latency

Table S6 evaluates every method under three further protocols. Paused physics shows how the relative performance changes when the cost of computation is ignored, equal latency compares all methods at the p95 latency of SE(3)-WAM, and the original executors check whether the shared executor removes a capability that a baseline relies on. With paused physics, the difference between SE(3)-WAM and [name] is [±X.X] points, and under equal latency it is [±X.X] points. The original executors change the SR of the baselines by at most [X.X] points. Table S7 gives the success at every added latency; its rows for SE(3)-WAM and the strongest baseline are the data of Fig. 1 (right).

Timing. Table S8 breaks the time-to-action of SE(3)-WAM on one RTX 5090 down by stage, measured as in Appendix B.3; Table 1 gives the p95 latency of every method and Table S4 the parameter counts. On the robot, we report the p50, p95, and maximum action age and the fraction of all queries of the test trials that miss the 50 ms deadline, together with the peak memory.

C.3. Results on DOMINO

On DOMINO [14], we compare SE(3)-WAM, $\pi_{0.5}$, DynamicWAM, and PUMA on 24 single-arm Franka tasks of its clean level-1 setting. All methods use the benchmark's demonstrations, scenes, and native evaluator, one training seed, and 50 episodes per task and method. We report SR and the manipulation score (MS), which scales route completion by penalties for leaving the workspace or view and for collisions (Table 2). SE(3)-WAM uses domain-specific normalization and reads the simulator pose of the target named by the instruction; slots without a valid pose are masked as in Appendix A.2. The RGB-based baselines do not receive this pose. [Summarize the DOMINO results.]

C.4. Final Ablations and Seed Stability

Table S10 reports the five controls of Sec. 4.3 (Appendix B.5) for each training seed, with the cross-seed mean and standard deviation and the paired difference from the full model, with native latency and with 150 ms of added latency. The last two columns repeat the paired differences without the phase-only wire passage (`t3_phase`). On development scenes, three training seeds of the same configuration differed by up to [7.4] points in SR, mostly on this task, whose success ranged from [0] to [71]% across seeds (Appendix B.4). The two task groups of Fig. 5 are listed in Appendix B.1. [Whether any conclusion of Sec. 4.3 changes for an individual seed or without `t3_phase`.]

C.5. Robustness to Pose Noise

With frozen weights, we evaluate three synthetic perturbations of the target pose: nominal temporally correlated noise, twice its amplitude, and nominal noise with contiguous missing observations. The nominal process combines a fixed per-episode bias and a stationary correlated component, each with per-axis standard deviations of 5 mm and 1°, with a correlation time of 200 ms. Dropouts last 200 ms, separated by geometrically distributed valid gaps with a mean of 3.8 s, giving a long-run missing fraction of 5%. The perturbation applies to the current and past estimates before the history features are computed.

Fig. S1 evaluates the final model under these perturbations, on the same held-out scenes at zero added latency and with all three training seeds. Relative to clean poses, SR changes by [X.X], [X.X], and [X.X] points under nominal correlated noise, twice its amplitude, and missing observations.

Figure placeholder: Fig. S1 — pose noise
x: clean poses, nominal correlated noise, twice its amplitude, and nominal noise with missing observations; y: SR of SE(3)-WAM per task family, frozen weights, real-time protocol; paired intervals over matched scenes.

Figure S1. **Robustness to pose noise.** SR of SE(3)-WAM under synthetic perturbations of the target pose, relative to clean poses.

C.6. Forecast Diagnostics

Table S9 compares the learned SE(3) forecast of the final model offline with constant-velocity and constant-acceleration extrapolation from the same pose history.

Table S5. **Per-task held-out success** (%) on Dynamic-14 for every method of Table 1, mean of three training seeds for our variants and one for baselines, real-time protocol. DP: Diffusion Policy. DVLA: DynamicVLA. DrVLA: DreamVLA. OWAM: OpenWAM-α. DWAM: DynamicWAM. +: with our SE(3) history tokens. σ: SE(3)-WAM with nominal synthetic correlated pose noise (Appendix C.5). Δ: SE(3)-WAM minus the strongest other method on that task, with its 95% paired interval. The last two rows give the position error at first contact e_p (mm), averaged over the 13 tasks with a contact, and the number n of episodes with a contact (Appendix B.4); –: no 13-task mean. Task codes are listed in Table S3.

Task	ACT	DP	$\pi_{0.5}$	$\pi_{0.5}$+	DVLA	PUMA	DrVLA	OWAM	DWAM	DWAM+	σ	**Ours**	Δ [95% CI]
Harmonic grasp	48.0	0.0	—	—	—	—	—	—	—	—	—	—	—
Conveyor grasp	46.0	0.0	—	—	—	—	—	—	—	—	—	—	—
Translating grasp	18.0	0.0	—	—	—	—	—	—	—	—	—	—	—
Rotating grasp	21.0	0.0	—	—	—	—	—	—	—	—	—	—	—
Occluded grasp	32.0	0.0	—	—	—	—	—	—	—	—	—	—	—
Moving receptacle	26.0	0.0	—	—	—	—	—	—	—	—	—	—	—
Rotating receptacle	0.0	0.0	—	—	—	—	—	—	—	—	—	—	—
Wire passage	99.0	0.0	—	—	—	—	—	—	—	—	—	—	—
Wire passage, full	0.0	0.0	—	—	—	—	—	—	—	—	—	—	—
Ball capture	36.0	0.0	—	—	—	—	—	—	—	—	—	—	—
Ball capture, table	71.0	0.0	—	—	—	—	—	—	—	—	—	—	—
Slider capture	57.0	0.0	—	—	—	—	—	—	—	—	—	—	—
Can capture	90.0	0.0	—	—	—	—	—	—	—	—	—	—	—
Post-impact grasp	44.0	0.0	—	—	—	—	—	—	—	—	—	—	—
Task-balanced	42.0	0.0	—	—	—	—	—	—	—	—	—	—	—
e_p (mm)	32.2	–	—	—	—	—	—	—	—	—	—	—	–
n	1109	6	—	—	—	—	—	—	—	—	—	—	–

Table S6. **Protocol controls** on held-out Dynamic-14 scenes (SR, %) under the protocols of Appendix B.3; real time is the protocol of Table 1. Late: queries in real time whose time-to-action exceeds the 50 ms control period (%). Age: p95 action age in real time (ms); –: no command executed.

Method	Paused	Equal latency	Original executor	Real time	Late (%)	Age p95
ACT	40.9	—	—	42.0	0.0	50
Diffusion Policy	—	—	—	0.0	100.0	–
$\pi_{0.5}$	—	—	—	—	—	—
$\pi_{0.5}$ + SE(3)	—	—	—	—	—	—
DynamicVLA	—	—	—	—	—	—
PUMA	—	—	—	—	—	—
DreamVLA	—	—	—	—	—	—
OpenWAM-α	—	—	—	—	—	—
DynamicWAM	—	—	—	—	—	—
DynamicWAM + SE(3)	—	—	—	—	—	—
SE(3)-WAM	—	—	–	—	—	—

Table S7. **Success versus added latency** on held-out Dynamic-14 scenes (SR in %, real-time protocol). The 0 and 150 ms columns equal the Native and +150 ms columns of Table 1.

	Added latency (ms)				
Method	0	50	100	150	200
ACT	42.0	—	—	24.1	—
Diffusion Policy	0.0	—	—	0.0	—
$\pi_{0.5}$	—	—	—	—	—
$\pi_{0.5}$ + SE(3)	—	—	—	—	—
DynamicVLA	—	—	—	—	—
PUMA	—	—	—	—	—
DreamVLA	—	—	—	—	—
OpenWAM-α	—	—	—	—	—
DynamicWAM	—	—	—	—	—
DynamicWAM + SE(3)	—	—	—	—	—
SE(3)-WAM	—	—	—	—	—

Table S8. **Latency breakdown** of one query of SE(3)-WAM on one RTX 5090 at batch size 1 (ms, p50 / p95), and action age on the robot.

Stage	p50	p95
Pose tracking (per frame, robot)	—	—
Image encoding	—	—
World stream (once per query)	—	—
Action stream (5 Euler steps)	—	—
Time-to-action (total)	—	—
Forecast heads (training only, for reference)	—	—
Action age at execution (robot)	—	—
Queries over 50 ms (robot, %)	—	

Table S9. **Offline prediction errors.** Target-position error at 400 ms (mm) on the same held-out observations. CV and CA are analytic extrapolations from causal pose history; the learned head reads frozen full-model states.

	Const. vel.	Harmonic	Roll/slide	Impact
CV extrapolation	—	—	—	—
CA extrapolation	—	—	—	—
Learned pose head	—	—	—	—

Table S10. **Final ablations by training seed** on held-out Dynamic-14 scenes (Appendix C.4). S0–S2: SR (%) of training seeds 0–2; mean ± s.d. over seeds; Δ: control minus SE(3)-WAM (points) with its paired 95% interval. Last two columns: Δ without the phase-only wire passage (`t3_phase`), 13 tasks.

	Native					+150 ms					Δ w/o `t3_phase`	
Variant	S0	S1	S2	Mean ± s.d.	Δ [95%]	S0	S1	S2	Mean ± s.d.	Δ [95%]	Native	+150 ms
SE(3)-WAM *	—	—	—	—	–	—	—	—	—	–	–	–
No target history	—	—	—	—	—	—	—	—	—	—	—	—
No future supervision	—	—	—	—	—	—	—	—	—	—	—	—
No scene-flow supervision	—	—	—	—	—	—	—	—	—	—	—	—
No dedicated SE(3)	—	—	—	—	—	—	—	—	—	—	—	—
Uniform-fidelity forecast	—	—	—	—	—	—	—	—	—	—	—	—

D. Real-Robot Experiments

D.1. Setup and Tasks

The robot is a Franka arm ([model]) with [gripper model] and a fixed RealSense D435 RGB-D camera, commanded through [a timestamped Cartesian interface] at [control frequency]. The camera records RGB-D at [resolution] and [frame rate], and we calibrate its extrinsics by [procedure] before each session. SE(3)-WAM uses this fixed third-person view; baselines whose recipes require a wrist view receive [per-method camera configuration]. All policies run on one RTX 5090. Fig. S2 shows the layout, and Table S11 defines the six tasks. T1 and T2 share a belt conveyor, T3 uses a turntable, T4 a remotely driven toy car, T5 a cue-driven collision between two balls, and T6 a ramp. Touching or pushing the target without completing the task, a single-finger hold, and an action after the target has left the workspace count as failures; [equipment-invalid trials and safety interventions are repeated or counted as failures by the following rule].

D.2. Pose Tracking and Demonstration Labels

Tracking is marker-free. Grounding DINO [25] localizes the object named by the task in the first frame, SAM 2.1 [32] gives its initial mask, and FoundationPose [44] initializes the object's 6D pose from this mask and then tracks it. [FoundationPose variant and mode (model-based with an object mesh or model-free with reference views), how the object frame is kept consistent across demonstrations and trials, and re-initialization after a loss.] The tracker runs on the control clock: at every control-period boundary, it updates the pose on the newest camera frame captured at or before the boundary, so the pose history has the 50 ms spacing used in training, and each estimate keeps the capture time of its frame. A query waits for the update of its own boundary and then runs the policy on the same GPU. While a query spans further boundaries, the tracker keeps updating at a higher GPU priority and processes only the newest frame, so that an update never queues behind older ones; the tracking time and this contention are part of the measured latency (Table S8). [Camera frame rate and the measured time per update.] When the target is lost, the estimate is marked invalid, and the previous pose is not carried forward. [How the target poses of the demonstrations, used as inputs and training labels, are obtained.] For the ball tasks, the target-state tokens keep the translation, velocity, and acceleration of the ball center and mask its orientation and angular motion, and the pose head is supervised on translation only. Table S12 reports the tracking accuracy.

D.3. Training and Trial Protocol

We compare SE(3)-WAM, $\pi_{0.5}$, DynamicWAM, and Diffusion Policy [12], with 20 trials per task and method. All methods use the same pre-generated trial keys, which fix the object instance, the initial pose, and the motion parameters, sampled by [sampling protocol]. Trials run in [randomized method order], so that drift of the apparatus does not align with the methods. An operator resets the scene. A trial starts at the first usable observation, the robot may move at any time, and a trial ends at success, failure, or a timeout of [X] s. Demonstrations, development trials, and test trials come from separate sessions. SE(3)-WAM is fine-tuned per task from the DROID base (Appendix A.4), and the baselines are trained or fine-tuned on the same demonstrations with [per-method initialization and training settings]. For every method, we log the end-to-end time-to-action of each query on the same RTX 5090, including tracking for SE(3)-WAM; Table 4 reports its p95.

D.4. Detailed Results and Failures

Table S13 reports successes per task, 95% binomial intervals for the pooled rates, and paired differences from a bootstrap over trial keys. [The most frequent failures of SE(3)-WAM and their causes.]

Figure placeholder: Fig. S2 — real-robot setup

(a) Left, photo of the workcell from behind the robot: the third-person RGB-D camera, the robot, and the conveyor, turntable, car track, ball-collision track, and ramp at their operating positions, each labeled; the wrist camera if a baseline uses one. **(b) Right, top-down schematic to scale:** the robot base frame with its axes, the camera position and viewing direction with its distance to the workspace, the footprint and motion direction of each device for T1–T6, the grasp or interception region of each task, and the workspace bounds in cm. **Rules:** real photo, unretouched; schematic from the calibrated extrinsics and measured device positions; no repetition of the task photos of Fig. 6.

Figure S2. **Real-robot setup.** (a) The workcell. (b) Top-down layout of the camera, the robot base frame, and the devices of T1–T6 [with the workspace dimensions].

Table S11. Real-robot task definitions. $N = 100$: training demonstrations per task; 20 additional demonstrations per task are reserved for validation.

Task	Device, motion, and objects	Operating range; N	Success and failure criteria
T1	Conveyor transports a graspable cube.	[belt speed]; 100	[stable grasp and lift; failures]
T2	Conveyor transports an open box; the arm places a cube into it.	[belt speed]; 100	[placement and retention; failures]
T3	Turntable carries a cube around its axis.	[angular speed, radius]; 100	[stable grasp and lift; failures]
T4	Remotely driven toy car moves on the tabletop.	[speed, path]; 100	[car grasp and lift; failures]
T5	A cue-driven ball strikes a second ball; the arm intercepts the second ball after the collision.	[impact and outgoing speed]; 100	[post-impact capture; failures]
T6	A ball rolls or a block slides down a ramp toward the interception region.	[slope, release, speed]; 100	[capture and retention; failures]

Table S12. **Real-robot pose tracking.** Median/p95 translation (mm) and rotation (°) error of the tracker against a marker-free reference, the uncertainty of that reference, and the fraction of lost observations. Ball rotation is not used.

Target	mm	°	Ref. uncertainty	Lost
Real rigid object	—	—	—	—
Real ball	—	–	—	—

Table S13. **Real-robot successes** per task (successes/trials, 20 trials per task; tasks in Table S11). All: pooled success rate (%) with its 95% binomial interval. Δ: paired difference from SE(3)-WAM (points) with a bootstrap interval over trial keys.

Method	T1	T2	T3	T4	T5	T6	All	Δ
$\pi_{0.5}$	—	—	—	—	—	—	—	—
DynamicWAM	—	—	—	—	—	—	—	—
Diffusion Policy	—	—	—	—	—	—	—	—
SE(3)-WAM	—	—	—	—	—	—	—	–

E. Design Studies

We study the history input, future supervision, and world–action interface with 4-layer models of width 768, trained for 4 epochs (40,104 updates at global batch size 16). The earlier synchronous executor executes one command per query on 1,400 development scenes, 100 per task; the initial forecast screen reports its 140-scene selection subset and the remaining 1,260 scenes separately. Encoders, inputs, and seed counts differ between studies; comparisons are within a study. The path, auxiliary, interface, and encoder studies retain the eight-frame visual windows and GT target-motion history of their screening recipe. The final-scale, real-time ablations are reported separately in Sec. 4.3 and Appendix C.4.

E.1. History Representation

Which history helps control? With a frozen DINOv3-L encoder [35] and one training seed, we compared seven visual-history encodings against the current image alone and against the current image with the target's SE(3) history (Table S14a). Every condition retains the current ground-truth target pose and the robot-state history. The visual encodings include flow-based alternatives [15, 28]. The target-history condition also receives past ground-truth poses and their derived kinematics.

Target history raises SR from 53.43% to 58.86%, a gain of 5.43 points with a paired 95% scene-bootstrap interval of $[3.07, 7.86]$. None of the tested visual histories exceeds the current-image reference in point estimate. More readable history features do not necessarily improve control: for the RGB window, a frozen linear probe of the summary representation reduces the error in reading position 350 ms in the past from 10.89 to 8.46 mm, while SR falls by 6.14 points. These results support retaining explicit target motion under this training recipe.

Does the trained policy use the history? We keep the target-history checkpoint fixed, replace its past target poses by the current pose, and recompute the kinematics. Current pose, image, robot state, and timestamps remain intact. SR falls from 58.86% to 0.64% (Table S14b), and the first executed action changes in all 210 diagnostic windows; an identity replacement reproduces the original actions. The drop measures sensitivity to incorrect history; the training benefit is the 5.43-point comparison above. The intervals quantify scene variation conditional on these single-seed checkpoints.

E.2. Future Representation and Supervision

Initial target screen. With frozen DINOv3-L and seed 0, we compared fifteen forecast configurations (Table S15). Geometry uses eight future horizons: target SE(3), base-frame displacements of a 14×14 patch grid, or projected 2D motion, all relative to the current state. Codec targets use current-to-future frame pairs; object DINO pools features inside the target mask. Wan2.2 VAE targets temporally compress the nine-frame clip, rather than providing one latent per horizon.

Table S14. **History input and test-time intervention.** Frozen DINOv3-L, one training seed, 1,400 scenes. All rows retain the current GT target pose and robot history. Δ is relative to the first row of each panel (points); brackets are pointwise paired 95% scene-bootstrap intervals, conditional on the trained checkpoints.

Condition	SR (%)	Δ [95% CI]
(a) Separately trained input variants		
Current image	53.43	ref.
RGB window	47.29	-6.14 $[-8.50, -3.79]$
Optical flow	51.43	-2.00 $[-4.43, +0.43]$
Flow + statistics	51.64	-1.79 $[-4.21, +0.64]$
Codec motion vectors	52.36	-1.07 $[-3.36, +1.21]$
Video-VAE latent	50.71	-2.71 $[-5.21, -0.14]$
Compressed window (age)	45.29	-8.14 $[-10.64, -5.64]$
Compressed window (uniform)	47.86	-5.57 $[-8.00, -3.14]$
Current image + SE(3) history	58.86	$+5.43$ $[+3.07, +7.86]$
(b) Same target-history checkpoint		
True history	58.86	ref.
Stationary history	0.64	-58.21 $[-59.86, -56.50]$

On the 1,260 remaining scenes, P07 and P09 have the highest point estimates, 64.05% and 64.13%, compared with 62.78% without future supervision (P00) and 62.38% for scene flow with SE(3) (P03). The paired comparisons do not establish an overall advantage for P07 or P09. Dense 2D displacement (P05) lowers SR by 3.57 points relative to P00 (Bonferroni-adjusted $p = 0.00844$ over 27 comparisons). These are comparisons of complete forecast configurations: P07 uses direct regression from shared features, whereas P03 uses an independent Future stream within the backbone and flow matching. Their difference therefore motivates a separate comparison of representation and prediction path.

Separating representation from prediction path. We crossed 2D and 3D targets with direct regression and the Future-stream flow-matching implementation, holding the trainable CNN, SE(3) supervision, current-target RGB reconstruction, and history inputs fixed, with three training seeds per condition (Table S16a). Both coordinate representations use the same source points and horizons; their scales are calibrated from training labels. Direct prediction gives higher SR than the Future-stream implementation in every seed for both 2D and 3D. Within either path, the 2D–3D difference changes sign across seeds. Across paths, the loss, motion weighting, and visibility supervision also change, so the result favors the complete direct-prediction recipe.

A further comparison fixes the direct path, frozen DINOv3-L, and all three future appearance targets (Ta-

Table S15. **Initial forecast-configuration screen.** Frozen DINOv3-L, seed 0. The 140-scene selection subset and the remaining 1,260 development scenes are disjoint. Rows are grouped by the changed target; prediction paths also differ, as described in Appendix E.2. A dash means that target is absent.

ID	Scene target	Object target	Additional target	Success / 140	SR_{1260} (%)
Basic supervision controls					
P00	–	–	–	88	62.78
P01	3D scene flow	–	–	95	62.06
P02	–	SE(3)	–	92	62.62
P03	3D scene flow	SE(3)	–	93	62.38
Scene-target alternatives to P03					
P04	Video-VAE latent	SE(3)	–	92	62.14
P05	Dense 2D displacement	SE(3)	–	87	59.21
P06	Codec motion vectors	SE(3)	–	86	63.10
P07	Sparse 2D tracks	SE(3)	–	90	64.05
Object-target alternatives to P03					
P08	3D scene flow	Object DINO	–	89	63.02
P09	3D scene flow	Target 2D tracks	–	88	64.13
Additional targets on P03					
A_α	3D scene flow	SE(3)	Motion magnitude	89	61.75
P12	3D scene flow	SE(3)	Depth	90	62.78
P13	3D scene flow	SE(3)	DINO	92	62.54
P14	3D scene flow	SE(3)	Dynamic RGB	89	62.70
P15	3D scene flow	SE(3)	Depth, DINO, dynamic RGB	94	63.65

Table S16. **Representation and prediction path.** SR (%), mean ± sample s.d. over three training seeds, each evaluated on 1,400 scenes. Both panels retain SE(3) and current-target RGB supervision; compare within a panel.

Prediction path	2D	3D
(a) CNN, no future appearance targets		
Direct regression	63.45 ± 2.94	64.19 ± 0.60
Future stream / flow matching	56.83 ± 5.41	58.19 ± 3.19
(b) Frozen DINOv3-L, all three appearance targets		
Direct regression	62.71 ± 1.65	62.21 ± 1.82

Table S17. **Future appearance supervision.** Frozen MAE-B, seed 1, 1,400 scenes; direct 3D flow, SE(3), and current-target RGB reconstruction stay enabled. Δ: SR difference from the first row (points). ✓: enabled; –: disabled.

Depth	DINO	Dynamic RGB	SR (%)	Δ
–	–	–	63.86	ref.
✓	–	–	61.93	−1.93
–	✓	–	67.57	+3.71
–	–	✓	66.14	+2.29
✓	✓	✓	71.07	+7.21

ble S16b). The 2D–3D difference is +2.00, −1.50, and +1.00 points across seeds, with a mean of +0.50. We retain 3D displacement to express scene motion in the robot's metric frame; these development studies do not establish a control-performance advantage of 3D over 2D.

Selecting auxiliary supervision. With frozen MAE-B and seed 1, we keep direct 3D flow, SE(3), and current-target RGB reconstruction, and compare no future appearance target, each target alone, and all three together (Table S17). DINO and dynamic RGB alone improve on the no-auxiliary reference, depth alone lowers SR, and the combination has the highest SR, 71.07%. The single-target rows measure the effect of adding that target to the reference; they do not isolate its marginal contribution when the other two are present. This screen motivates retaining the combination with MAE-B. Auxiliary benefits depend on the encoder: in the three-seed CNN study, all three targets average 62.86%, versus 63.95% with none. The final-model no-future control in Sec. 4.3 tests the full forecast supervision set, including the geometric targets.

E.3. World–Action Information Flow

Interfaces and controls. With frozen MAE-B and shared attention with separate world/action feed-forward networks, we distinguish observed conditions C, future-query hidden states W, and decoded predictions Y. Actions always read C. One object query and one query per patch read C and

Table S18. **World–action information flow.** Frozen MAE-B, three training seeds, 1,400 development scenes per seed. W: future-query hidden states; A: action states; Y: decoded geometry. Actions always read observed conditions C. R1–R4 form the two-edge comparison. SR is the seed mean ± sample s.d. (%); Δ_{R1} is the mean ± sample s.d. of same-seed differences from R1 (points).

ID	Future losses	$W \to A$	$A \to W$	$Y \to A$	SR (%)	Δ_{R1}
R0	–	–	–	–	60.45 ± 3.57	-3.14 ± 5.75
R1	✓	–	–	–	63.60 ± 2.34	ref.
R2	✓	✓	–	–	62.52 ± 1.97	-1.07 ± 4.28
R3	✓	–	✓	–	64.21 ± 2.13	$+0.62 \pm 0.84$
R4	✓	✓	✓	–	63.00 ± 4.19	-0.60 ± 6.52
R5	✓	–	–	✓	63.95 ± 1.91	$+0.36 \pm 3.45$

one another to predict SE(3) and scene flow. R1–R4 keep the modules, losses, and initialization procedure fixed and toggle the two attention edges $W \to A$ and $A \to W$ (Table S18). Their recorded joint evaluation recomputes W at each Euler step. R0 keeps the same modules but disables all future losses; current RGB reconstruction remains. R5 instead decodes Y once per query and re-encodes the trajectory, flow, and visibility into action conditions with a 0.37M-parameter geometry encoder. It uses predicted geometry during both training and inference, with action gradients flowing through that interface.

Supervision and action conditioning. R1 improves on R0 by 3.14 points on average, with seed-wise differences of -1.29, $+9.64$, and $+1.07$. Adding hidden-state access (R2) or decoded geometry (R5) changes SR relative to R1 by -1.07 and $+0.36$ points, respectively, with seed-wise difference standard deviations of 4.28 and 3.45 points. Neither produces a consistent gain across seeds. R3 has the highest mean, but its 0.62-point gain over R1 becomes -0.03 points outside the phase-only wire task; all fourteen tasks remain in the primary metric. The main effects of the two edges and their interaction also change sign across seeds.

Relation to the deployed model. We retain future supervision without a forecast-to-action interface. R1 uses independent future queries; SE(3)-WAM reads forecasts directly from the existing scene and summary tokens (Sec. 3.2). These distinct architectures share the supervision-only use of forecasts. In SE(3)-WAM, actions still read the observed world representation, while forecast heads are skipped at deployment.

E.4. Encoder and Parameter Sharing

Observation encoder. We compare a trainable CNN with frozen DINOv3-S, DINOv3-L [35], and MAE-B [17], keeping direct 3D prediction, SE(3), and the three appearance targets (Table S19a). Each uses three seeds and the same 8-GPU training recipe and global batch size. MAE-B has the highest mean SR, 66.76%, and is selected for the subsequent interface and backbone studies.

Table S19. **Encoder and parameter sharing.** Three seeds, 1,400 scenes per seed; direct 3D, SE(3), and all appearance targets. SR: seed mean ± sample s.d. (%). Panel (b) fixes MAE-B, reusing (a)'s shared-attention runs. Params: trainable backbone; ⋆: selected.

Configuration	Status / params	SR (%)
(a) Observation encoder		
CNN	trainable	62.86 ± 2.64
DINOv3 ViT-S/16	frozen	61.69 ± 2.00
DINOv3 ViT-L/16	frozen	62.21 ± 1.82
MAE ViT-B/16⋆	frozen	66.76 ± 3.84
(b) World/action parameter sharing		
Shared attention⋆	47.266M	66.76 ± 3.84
Separate parameters	47.272M	62.86 ± 2.98

Parameter sharing. With MAE-B fixed, we compare shared attention and stream-specific feed-forward networks against separate world/action attention and feed-forward parameters (Table S19b). Both keep the same information flow and about 47.3M trainable backbone parameters; the separate variant uses a smaller feed-forward width to match this count. Shared attention averages 66.76% SR, against 62.86% for separate parameters. Their mean gap falls to 0.39 points when both wire tasks are excluded, so the difference is concentrated in those tasks. We retain shared attention based on this aggregate comparison.